RV College of Engineering®
Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

Go, change the world

Design and Analysis of Oversampling ADC

A Minor Project Report (XX367P)

Submitted by,

P Narashimaraja	**1RV22EC005**
Nithin M	**1RV22EC006**
Aditya	**1RV22EE007**
Bala N	**1RV22EC007**

Under the guidance of

Mr. Ramavenkateshwaran
Assistant Professor
Dept. of ECE
RV College of Engineering

Dr. Ramavenkateshwaran
Assistant Professor
Dept. of ECE
RV College of Engineering

In partial fulfillment of the requirements for the degree of Bachelor of Engineering in

Electronics and Communication Engineering

2024-25

RV College of Engineering®, Bengaluru

(*Autonomous institution affiliated to VTU, Belagavi*)

Department of Electronics and Communication Engineering

CERTIFICATE

Certified that the minor project (XX367P) work titled ***Design and Analysis of Oversampling ADC*** is carried out by **P Narashimaraja** (**1RV22EC005**), **Nithin M** (**1RV22EC006**), **Aditya** (**1RV22EE007**) and **Bala N** (**1RV22EC007**) who are bonafide students of RV College of Engineering, Bengaluru, in partial fulfillment of the requirements for the degree of **Bachelor of Engineering** in **Electronics and Communication Engineering** of the Visvesvaraya Technological University, Belagavi during the year 2024-25. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the minor project report deposited in the departmental library. The minor project report has been approved as it satisfies the academic requirements in respect of minor project work prescribed by the institution for the said degree.

Guide	Head of the Department	Principal
Mr. Ramavenkateshwaran	Dr. Ravish Aradhya	Dr. K. N. Subramanya

External Viva

Name of Examiners	Signature with Date
1.	
2.	

DECLARATION

We, **P Narashimaraja**, **Nithin M**, **Aditya** and **Bala N** students of sixth semester B.E., Department of Electronics and Communication Engineering, RV College of Engineering, Bengaluru, hereby declare that the minor project titled '**Design and Analysis of Oversampling ADC**' has been carried out by us and submitted in partial fulfilment for the award of degree of **Bachelor of Engineering** in **Electronics and Communication Engineering** during the year 2024-25.

Further we declare that the content of the dissertation has not been submitted previously by anybody for the award of any degree or diploma to any other university.

We also declare that any Intellectual Property Rights generated out of this project carried out at RVCE will be the property of RV College of Engineering, Bengaluru and we will be one of the authors of the same.

Place: Bengaluru

Date:

	Name	Signature
1.	P Narashimaraja(1RV22EC005)	
2.	Nithin M(1RV22EC006)	
3.	Aditya(1RV22EE007)	
4.	Bala N(1RV22EC007)	

ACKNOWLEDGEMENTS

We are indebted to our guide, **Mr. Ramavenkateshwaran**, Assistant Professor, Department of ECE, RVCE for the wholehearted support, suggestions and invaluable advice throughout our project work and also helped in the preparation of this thesis.

We also express our gratitude to our panel member **Prof. Rajesh R. M.**, Assistant Professor, Department of AIML, RVCE for his invaluable comments and suggestions during the phase evaluations.

Our sincere thanks to the project coordinators **Prof. Rushikesh Anil Padaki**, Assistant Professor, Department of AIML and **Prof. Harshitha V**, for their timely instructions and support in coordinating the project.

Our gratitude to **Prof. Narashimaraja P**, Department of ECE, RVCE for the organized latex template which made report writing easy and interesting.

Our sincere thanks to **Dr. Ravish Aradhya**, Professor and Head, Department of Electronics and Communication Engineering, RVCE for the support and encouragement.

We express sincere gratitude to our beloved Professor and Vice Principal, **Dr. Geetha K S**, RVCE and Principal, **Dr. K. N. Subramanya**, RVCE for the appreciation towards this project work.

We thank all the teaching staff and technical staff of Electronics and Communication Engineering department, RVCE for their help.

Lastly, we take this opportunity to thank our family members and friends who provided all the backup support throughout the project work.

ABSTRACT

The healthcare sector generates vast quantities of unstructured clinical data through documents such as referral letters and discharge summaries. Manual processing of these records is time-consuming and error-prone, creating significant bottlenecks in clinical workflows. This project addressed the challenge of automated medical document understanding and Systematized Nomenclature of Medicine — Clinical Terms (SNOMED CT) coding within a template-free intelligent extraction framework.

The domain of clinical Natural Language Processing (NLP) has expanded rapidly with the adoption of Electronic Health Records (EHR) and the growing volume of clinical documents requiring structured coding. However, many existing systems rely on rigid, template-dependent approaches that fail to adapt to diverse document layouts commonly encountered in real-world National Health Service (NHS) environments. This limitation often results in incomplete or inaccurate code mappings.

The proposed system, developed as an Intelligent Clinical Document Processing System (ICDPS), aimed to: (i) create an intelligent ingestion module for PDF and image-based clinical documents; (ii) develop a non-template-based extraction engine for diverse layouts; (iii) automate clinical summarization and categorization into predefined medical fields; and (iv) normalize free-text clinical concepts to SNOMED CT codes.

The methodology followed an iterative Software Development Life Cycle (SDLC) consisting of requirements analysis, design, implementation, testing, and deployment. OCR pipelines handled image-based inputs, while direct extraction processed PDFs. Transformer-based NLP models were employed for Named Entity Recognition (NER) and classification of entities such as Diagnosis, Medication, Procedure, and Allergy. A confidence-ranked SNOMED CT suggestion interface assisted users in selecting suitable codes.

The system achieved clinical entity extraction accuracy exceeding 88% and SNOMED CT mapping precision of 85% on the test dataset, satisfying project requirements. The resulting system demonstrated potential to reduce administrative burden, accelerate structured record generation, and improve the quality of medical coding in clinical environments.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

Chapter 1

Introduction

CHAPTER 1

INTRODUCTION

Clinical documents form a large part of day-to-day healthcare operations. Referral letters, discharge summaries, laboratory reports, outpatient records, and clinical notes are routinely created during patient care and contain information needed for diagnosis, treatment planning, follow-up decisions, and administrative activities. Although these documents are rich in clinical information, much of the content exists as unstructured text, making large-scale analysis and automated processing difficult.

During discussions with healthcare practitioners and through examination of NHS clinical records, it became apparent that a significant amount of useful information remains embedded within narrative documents rather than being available in a structured format. Retrieving specific clinical details often requires manual review, particularly when information is spread across multiple documents or presented in different layouts.

The widespread adoption of Electronic Health Records (EHRs) has increased the availability of digital clinical data, but it has not eliminated the challenges associated with processing clinical documents. Healthcare organizations continue to manage documents originating from different systems, specialties, and workflows. As document volumes increase, manual extraction and coding become increasingly time-consuming and place additional demands on clinical and administrative staff.

Recent advances in Natural Language Processing (NLP), transformer-based language models, and Optical Character Recognition (OCR) have made automated clinical document processing more practical than in previous years. Modern language models can identify clinically relevant concepts from free text, while OCR systems can convert scanned documents and image-based records into machine-readable text. Together, these technologies provide an opportunity to automate parts of the clinical information extraction workflow.

This project investigates how these technologies can be combined within a single processing framework. The resulting Intelligent Clinical Document Processing System (ICDPS) processes both PDF and image-based clinical documents and integrates OCR, document understanding, clinical entity extraction, and SNOMED CT terminology mapping. The objective is to transform unstructured clinical information into structured outputs that can support clinical coding, document review, and healthcare information

management activities.

This chapter introduces the problem domain, outlines the motivation for the work, defines the project objectives and scope, reviews relevant literature, identifies research gaps, and presents the methodology adopted during system development.

1.1 Introduction

Processing clinical documents remains a practical challenge within many healthcare environments. Referral letters, discharge summaries, diagnostic reports, prescriptions, and specialist correspondence are generated continuously and contain information required for both patient care and administrative processes. While these documents are increasingly stored digitally, much of their content remains unstructured and therefore difficult to process automatically.

A review of NHS clinical documents used during this project highlighted the diversity of document formats encountered in practice. Some documents contained selectable digital text, whereas others were available only as scanned PDFs or image-based records. Layouts varied considerably between document sources, making template-based extraction approaches difficult to apply consistently.

Clinical coding and information extraction are still heavily dependent on manual review in many workflows. Human review remains valuable for interpreting complex medical information, but it requires significant time and effort, particularly when large volumes of documents must be processed. Variability in interpretation can also affect coding consistency across different users and organizations.

Advances in transformer-based language models have improved the ability of NLP systems to extract information from clinical text. Models such as BioBERT and ClinicalBERT have demonstrated strong performance on medical entity extraction tasks, enabling automated identification of diseases, medications, procedures, symptoms, and other clinically relevant concepts. At the same time, improvements in OCR technology have increased the feasibility of processing scanned and image-based clinical records.

The work presented in this project focuses on the development of a template-independent framework for clinical document processing and SNOMED CT coding. The proposed system combines OCR-based text extraction, clinical entity recognition, information categorization, and terminology mapping within a unified processing pipeline. By integrating these components, the system aims to reduce manual processing effort and provide struc-

tured representations of information contained within heterogeneous clinical documents.

1.1.1 Terminology and Definitions

The development of the Intelligent Clinical Document Processing System (ICDPS) involves several concepts and technical terms from healthcare informatics, Natural Language Processing (NLP), Artificial Intelligence (AI), and clinical coding systems. Understanding these terms is essential for interpreting the design and implementation aspects of the proposed system. The important terminology used throughout this report is presented below.

- **Clinical Document:** A healthcare-related document containing patient information, medical history, diagnoses, prescriptions, laboratory reports, referrals, discharge summaries, and clinical observations.
- **Natural Language Processing (NLP):** A branch of Artificial Intelligence concerned with enabling computers to understand, process, analyze, and generate human language.
- **Optical Character Recognition (OCR):** A technology used for converting text embedded within scanned documents or images into machine-readable textual information.
- **Named Entity Recognition (NER):** A Natural Language Processing task used to identify and classify specific entities from text into predefined categories such as diseases, medications, procedures, symptoms, and patient details.
- **SNOMED CT:** Systematized Nomenclature of Medicine – Clinical Terms, a comprehensive clinical terminology system used to represent medical concepts in a standardized manner.
- **Electronic Health Record (EHR):** A digital version of a patient's medical history that includes clinical data collected and maintained across healthcare systems.
- **Transformer Model:** A deep learning architecture that uses attention mechanisms to capture contextual relationships within sequential data and is widely used in modern NLP systems.

- **Intelligent Clinical Document Processing System (ICDPS):** The proposed framework designed to automate extraction of clinical information from heterogeneous healthcare documents and map extracted information to standardized SNOMED CT codes.

1.2 Overview of the Project Topic

Clinical documents contain a large amount of information that is essential for patient care, including diagnoses, medications, laboratory results, procedures, and clinical observations. Although this information is routinely recorded within healthcare systems, much of it remains embedded in unstructured text documents. Extracting clinically meaningful information from these documents is a challenging task that requires techniques from healthcare informatics, natural language processing, and machine learning.

The development of modern NLP models has made it possible to identify medical concepts, classify clinical information, and support standardized clinical coding, all automatically. These advances provide new opportunities for automating clinical document processing, but practical challenges remain in extracting, organizing, and standardizing information from heterogeneous healthcare records. The following subsections discuss the broader context of the problem and the factors that motivated the development of the proposed system.

1.2.1 Global Scenario

Healthcare systems worldwide are generating larger volumes of digital clinical information than ever before. Referral letters, discharge summaries, outpatient reports, laboratory results, and clinical notes are routinely created and stored electronically, resulting in substantial collections of healthcare documents that must be reviewed, managed, and interpreted.

The growth of digital records has increased interest in clinical natural language processing and automated information extraction. Early clinical NLP systems relied heavily on handcrafted rules, keyword dictionaries, and predefined document templates. While these approaches performed well on documents that closely matched their design assumptions, performance often declined when applied to records with different layouts, terminology, or writing styles.

Recent research has shifted towards transformer-based language models that learn

contextual representations directly from clinical text. Models such as BioBERT [1] and ClinicalBERT [2] have achieved strong results on tasks including clinical named entity recognition, concept extraction, and biomedical text understanding. These advances have made large-scale processing of clinical documents more practical than was previously possible.

Alongside developments in NLP, healthcare organizations have increased their focus on interoperability and standardized clinical terminology. Standards such as SNOMED CT support consistent representation of medical concepts across different healthcare systems and organizations. As a result, there is growing interest in solutions that can extract information from unstructured clinical documents and convert it into standardized, reusable formats.

1.2.2 Societal Relevance

The impact of clinical document processing extends beyond technical efficiency and affects multiple groups involved in healthcare delivery.

For patients, important clinical information is often distributed across referral letters, discharge summaries, investigation reports, and consultation records. Ensuring that relevant information can be identified and recorded consistently helps reduce the risk of incomplete documentation and improves the availability of information during future clinical encounters. Structured clinical records also simplify information sharing when patients receive care from multiple healthcare providers.

For healthcare professionals, reviewing large numbers of clinical documents is a routine but time-consuming activity. Information extraction and clinical coding frequently require manual examination of lengthy documents before relevant findings can be recorded. Automating repetitive extraction tasks can reduce administrative effort and allow clinicians and coding staff to spend more time on activities that require clinical expertise and decision-making.

Healthcare organizations also rely heavily on structured clinical information. Coded data is used for service reporting, audit activities, resource allocation, quality improvement programmes, and clinical research. Consistent use of standardized terminologies such as SNOMED CT improves interoperability between healthcare systems and reduces ambiguity when information is exchanged between organizations. As healthcare records continue to grow in volume, tools that assist with extracting and standardizing informa-

tion are becoming increasingly relevant to routine healthcare operations.

1.2.3 Problem Area

During the development of this project, it became apparent that clinical documents present several challenges that are not easily addressed by conventional information-extraction approaches. Although significant progress has been made in clinical NLP, many real-world healthcare documents remain difficult to process automatically because of differences in format, content structure, and terminology usage.

One of the most noticeable issues is document variability. The clinical records examined for this project included referral letters, discharge summaries, laboratory reports, scanned correspondence, and image-based documents originating from different healthcare workflows. Document layouts varied considerably, and information was not always presented in a consistent location or format. As a result, approaches that depend heavily on predefined templates or fixed document structures are often difficult to apply across a diverse document collection.

Clinical language introduces an additional layer of complexity. The same medical concept may be expressed using abbreviations, synonyms, partial descriptions, or specialty-specific terminology. Furthermore, the meaning of a term frequently depends on its surrounding context. Correctly identifying a clinical concept and linking it to an appropriate SNOMED CT code therefore requires contextual understanding rather than simple keyword matching.

A further limitation observed in many existing solutions is their reliance on digitally generated text. In practice, healthcare organizations continue to manage scanned referral letters, archived records, and image-based documents that must first undergo OCR processing before information extraction can take place. OCR errors, variations in image quality, and differences in document layout can all affect the quality of downstream NLP tasks.

These factors create a workflow in which document understanding, information extraction, and clinical coding cannot be treated as independent problems. Reliable processing requires a combination of OCR, contextual language understanding, and terminology mapping working together within a single framework. This requirement motivated the development of the Intelligent Clinical Document Processing System (ICDPS), which integrates these components into a unified processing pipeline for heterogeneous clinical

documents.

1.3 Specific Details of the Project Topic

The Intelligent Clinical Document Processing System (ICDPS) was developed to process heterogeneous clinical documents and convert their contents into structured, clinically meaningful information. The system was designed as a modular pipeline in which each stage performs a specific processing task before passing its output to the next component.

Processing begins with document ingestion. Clinical documents may be supplied as digitally generated PDFs, scanned reports, referral letters, or image-based records. During development, it was observed that a single extraction approach was not suitable for all document types. Documents containing embedded text were processed directly, whereas scanned and image-based documents required OCR before further analysis could be performed. The output of this stage is a normalized text representation that serves as the input for downstream processing.

The next stage focuses on clinical information extraction. This component identifies clinically relevant concepts from document text, including diagnoses, medications, procedures, anatomical references, allergies, laboratory findings, and temporal expressions. Extracting these entities is a critical step because subsequent processing stages depend on the accuracy and completeness of the identified clinical information.

Once entities have been extracted, the information is organized into categories that support document review and interpretation. Grouping related findings allows important information to be presented in a more structured manner than the original free-text document. During system design, this categorization stage was introduced to reduce the effort required to locate clinically relevant information within lengthy documents.

The system also generates concise summaries of document content. Rather than requiring users to review an entire document, the summarization stage produces a condensed representation that highlights the most relevant clinical information identified during extraction. This functionality was included to support faster review of large clinical records while preserving important findings.

A further objective of the project was to support standardized clinical coding. To achieve this, extracted entities are processed through a SNOMED CT mapping module. Candidate concepts are retrieved using semantic similarity search and then ranked using

contextual information derived from the source document. The resulting output provides confidence-ranked coding suggestions that can be reviewed and validated by healthcare professionals.

By combining document ingestion, information extraction, categorization, summarization, and terminology mapping within a single workflow, the ICDPS aims to reduce the amount of manual effort required to process clinical documents while improving the consistency and accessibility of extracted information.

1.4 Purpose and Motivation

The motivation for this project originated from the practical difficulties associated with processing large volumes of clinical documentation. Referral letters, discharge summaries, investigation reports, and other clinical records contain information that is important for patient care, yet much of this information remains embedded within unstructured text. Extracting relevant details from these documents often requires manual review, particularly when records vary in format, length, and presentation.

During the initial investigation of existing workflows, it became evident that document review and clinical coding involve a substantial amount of repetitive information extraction. Healthcare professionals frequently need to identify diagnoses, medications, procedures, and other clinical findings before these details can be recorded or coded. This observation motivated the development of a system capable of assisting with these tasks by automatically identifying clinically relevant information from document text.

A further objective was to explore whether recent advances in clinical NLP could be applied within an end-to-end document processing workflow. Rather than replacing clinical judgement, the intention was to provide structured information and coding recommendations that could support faster document review and reduce manual effort associated with routine processing tasks.

The project was also motivated by the challenges associated with terminology standardization. Mapping free-text clinical descriptions to SNOMED CT concepts is not always straightforward because multiple concepts may appear relevant depending on context. Providing confidence-ranked coding suggestions offered an opportunity to assist users in selecting appropriate concepts while retaining human oversight of the final coding decision.

From a technical perspective, the availability of biomedical language models, clinical

NLP research datasets, and established evaluation benchmarks provided an opportunity to investigate modern document-processing techniques within an academic setting. These resources enabled the design, implementation, and evaluation of a complete clinical document processing pipeline capable of handling document ingestion, information extraction, summarization, and SNOMED CT mapping within a unified framework.

1.5 Problem Statement

Clinical information is routinely recorded in referral letters, discharge summaries, laboratory reports, and other narrative healthcare documents. Although these documents contain information required for clinical care, coding, and record management, much of the content remains embedded within unstructured text. Extracting relevant information therefore often requires manual review, particularly when documents differ in format, layout, and presentation.

A review of representative NHS clinical documents used during this project highlighted several practical difficulties. Document structures varied considerably between sources, scanned and image-based records required OCR before analysis could be performed, and clinical concepts were frequently expressed using abbreviations, synonyms, and context-dependent terminology. These factors reduced the effectiveness of approaches based solely on predefined templates, document-specific rules, or keyword matching.

The problem addressed in this project is the development of a template-independent clinical document processing system capable of handling heterogeneous PDF and image-based documents, extracting clinically relevant information, generating structured summaries, and producing confidence-ranked SNOMED CT coding suggestions. The system is intended to reduce manual processing effort while supporting consistent representation of clinical information through standardized terminology mapping.

1.6 Objectives

The objectives of the project are:

1. To implement a document ingestion framework that processes both digital PDFs and image-based clinical documents, automatically selecting direct text extraction or OCR processing according to document characteristics.

2. To develop a BioBERT-based clinical entity extraction module capable of identify-

ing and classifying clinically relevant information, including diagnoses, medications, procedures, allergies, laboratory findings, and anatomical references from unstructured clinical text.

3. To generate structured summaries of clinical documents and organize extracted information into role-specific review categories.

4. To develop a SNOMED CT mapping module that retrieves and ranks candidate concepts using semantic similarity techniques and contextual relevance scoring.

5. To achieve accurate terminology mapping by providing confidence-ranked SNOMED CT code suggestions for extracted clinical entities.

6. To evaluate the complete end-to-end system using representative clinical documents and standard NLP evaluation metrics.

1.7 Scope and Relevance

This project focuses on the design, implementation, and evaluation of an Intelligent Clinical Document Processing System (ICDPS) for extracting structured information from English-language clinical documents. The system combines document processing, clinical natural language processing, and terminology mapping techniques to transform unstructured clinical text into structured outputs that can be used for downstream healthcare applications.

The implementation supports multiple document formats, including digitally generated PDF files, scanned documents, and image-based records such as JPEG, PNG, and TIFF files. Text is extracted using either direct PDF parsing or OCR, depending on the document type. The extracted content is then processed through a clinical NLP pipeline that performs entity extraction, information categorization, summarization, and SNOMED CT mapping. The final output is generated in a structured machine-readable format, allowing extracted information and coding suggestions to be reviewed by end users before acceptance.

The scope of the project is limited to clinical document processing, information extraction, summarization, and SNOMED CT terminology mapping. The system was not designed to perform medical diagnosis, recommend treatments, or provide clinical decision support. Integration with live Electronic Health Record systems, real-time patient

monitoring platforms, and large-scale healthcare infrastructure was not included within the implementation developed for this project. Similarly, support for non-English clinical documents and highly complex handwritten records was not investigated.

Development and evaluation were conducted within a controlled experimental environment. The implemented workflow processes documents in batch mode, allowing individual pipeline components to be developed, tested, and evaluated independently. Real-time processing services, enterprise-scale deployment, and integration with operational healthcare systems remain areas for future work.

The system also assumes that input documents are of sufficient quality for OCR and downstream NLP processing. During testing, document quality was found to influence extraction performance, particularly for scanned records containing image artefacts, skew, low contrast, or unusual layouts. As a result, overall system performance depends on both document quality and the accuracy of preceding processing stages.

Despite these limitations, the project demonstrates how OCR, clinical information extraction, document summarization, semantic retrieval, and SNOMED CT mapping can be integrated within a single processing framework. The resulting system provides a practical demonstration of how unstructured clinical documents can be transformed into structured information suitable for review and coding workflows. The techniques explored are also relevant to broader research areas including clinical NLP, healthcare informatics, and intelligent document understanding.

1.8 Literature Review

Before designing the proposed system, a review of existing research was conducted to understand how clinical document processing, biomedical NLP, information extraction, and clinical terminology mapping have been addressed in previous studies. The review focused on identifying techniques that could be applied to heterogeneous clinical documents as well as limitations that remain unresolved in current approaches.

Particular attention was given to research on clinical named entity recognition, transformer-based biomedical language models, OCR-assisted document processing, semantic retrieval, ontology mapping, and healthcare information systems. These areas form the core technical components of the proposed ICDPS framework and directly influenced the selection of methods used during implementation.

The literature review also helped identify recurring challenges reported across multiple

studies, including variability in document structure, dependence on domain-specific annotations, terminology ambiguity, and difficulties associated with processing scanned or image-based clinical records. Understanding these limitations was important in defining the scope and objectives of the project.

1.8.1 Literature Survey

Table **??** presents representative studies relevant to the development of the proposed system. The selected papers cover a range of topics including biomedical language models, clinical entity extraction, OCR-based document processing, semantic retrieval, ontology mapping, relation extraction, de-identification, explainable AI, and healthcare information systems.

Each study was reviewed to understand its underlying methodology, reported performance, practical advantages, and identified limitations. Rather than focusing on a single aspect of clinical NLP, the survey was used to examine how different technologies could contribute to an end-to-end clinical document processing workflow.

Several common observations emerged from the review. Many studies reported strong performance on narrowly defined tasks such as entity extraction or document classification, but fewer addressed the complete workflow from document ingestion to standardized terminology mapping. Support for heterogeneous document formats, integration of OCR with downstream NLP processing, and reliable mapping of extracted concepts to clinical terminologies remained less extensively explored.

These observations influenced a number of design decisions adopted in the proposed system, including the use of transformer-based language models for information extraction, OCR support for image-based documents, and semantic retrieval techniques for SNOMED CT concept mapping.

Table 1.1: Literature Survey of Representative Research Works

Ref	Author(s)	Title	Year	Methodology	Key Findings	Advantages	Limitations	Relevance to Project
[1]	Lee J. et al.	BioBERT: A Pre-trained Biomedical Language Representation Model for Biomedical Text Mining	2020	Pre-trained BERT on biomedical corpora (PubMed and PMC) and fine-tuned on biomedical NLP tasks	Domain-specific pretraining significantly improved biomedical NER and relation extraction performance	High contextual understanding of biomedical terms; state-of-the-art performance	Requires large computational resources and extensive training data	Forms the primary NER backbone for the proposed ICDPS system
[2]	Alsentzer E. et al.	Publicly Available Clinical BERT Embeddings	2019	Fine-tuned BERT on MIMIC-III clinical notes	Clinical-specific training improved understanding of healthcare text	Better contextual representation for clinical language	Limited by availability of domain-specific clinical datasets	Used as an alternative NER backbone during model comparison
[3]	Spasic I., Nenadic G.	Clinical Text Data in Machine Learning: Systematic Review	2020	Systematic review of 67 clinical text mining studies	Identified data scarcity and annotation challenges	Comprehensive review of clinical NLP limitations	No experimental implementation	Guided transfer learning and data augmentation approaches
[4]	Stubbs A., Uzuner O.	Annotating Longitudinal Clinical Narratives for De-identification	2015	Clinical document annotation for PHI detection	Established benchmark dataset for clinical NER evaluation	Standardized annotation framework	Focused primarily on de-identification tasks	Influenced entity categorization in the proposed system
[5]	Suominen H. et al.	Overview of the ShARe/CLEF eHealth Evaluation Lab 2013	2013	Shared clinical NER task evaluation	Best systems achieved high disorder recognition accuracy	Provides benchmark datasets	Limited dataset diversity	Used as baseline for model performance comparison
[6]	Savova G.K. et al.	Mayo Clinical Text Analysis and Knowledge Extraction System (cTAKES)	2010	Rule-based and NLP-based clinical information extraction	Effective extraction of clinical concepts	Open-source and modular architecture	Lower adaptability to unseen patterns	Used as baseline extraction framework
[7]	Lample G. et al.	Neural Architectures for Named Entity Recognition	2016	BiLSTM-CRF with character embeddings	Achieved state-of-the-art NER performance	Handles sequence dependencies effectively	Computationally slower than transformer models	Baseline NER model for comparison
[8]	Devlin J. et al.	BERT: Pre-training of Deep Bidirectional Transformers	2019	Transformer pretraining with MLM and NSP tasks	Improved contextual language understanding	Strong transfer learning capability	High training cost	Foundation of BioBERT and ClinicalBERT
[9]	Boag W. et al.	CliNER 2.0: Accessible and Accurate Clinical Concept Extraction	2018	Dictionary lookup with deep learning methods	Competitive concept extraction performance	User-friendly implementation	Reduced performance on complex entities	Inspired extraction interface design
[10]	Liu Y. et al.	RoBERTa: A Robustly Optimized BERT Pretraining Approach	2019	Optimized BERT training with larger batches	Improved downstream task performance	Better language representations	Higher computational requirements	Used for ablation studies
[11]	M. Topaz, L. Shafran-Topaz, and K. H. Bowles	I-MeDeSA: A Mobile Application for Point-of-Care Standardized Nursing Documentation	2013	Mobile application integrating SNOMED CT terminology and rule-based mapping for nursing documentation	Demonstrated feasibility of converting free-text nursing inputs into standardized concepts	Supports standardized healthcare documentation; portable clinical use	Limited to nursing-specific scenarios	Helps design standardized terminology mapping and user interaction modules
[12]	C. Jonquet, N. Shah, and M. Musen	The Open Biomedical Annotator	2009	Ontology-based annotation using biomedical terminology repositories and dictionary matching	Enabled automatic identification and annotation of biomedical concepts	Supports multiple biomedical ontologies	Performance dependent on ontology coverage	Guides ontology lookup and candidate retrieval for SNOMED mapping

Continued on next page

Table 1.1 (Continued)

Ref	Author(s)	Title	Year	Methodology	Key Findings	Advantages	Limitations	Relevance to Project
[13]	S. Zheng et al.	Joint Entity and Relation Extraction Based on a Hybrid Neural Network	2017	CNN and BiLSTM hybrid neural architecture for simultaneous extraction	Joint extraction improved relationship prediction accuracy over pipeline methods	Reduces error propagation between tasks	Increased computational complexity	Supports simultaneous NER and relation extraction
[14]	O. Uzuner, B. R. South, S. Shen, and S. L. DuVall	2010 i2b2/VA Challenge on Concepts, Assertions, and Relations in Clinical Text	2011	Shared-task benchmark using annotated clinical narratives	Top systems achieved strong concept extraction performance	Standard benchmark dataset and evaluation framework	Dataset size and diversity limitations	Used for fine-tuning and evaluating extraction models
[15]	W. W. Chapman, W. Bridewell, P. Hanbury, G. F. Cooper, and B. G. Buchanan	A Simple Algorithm for Identifying Negated Findings and Diseases in Discharge Summaries	2001	Rule-based negation detection using trigger terms (NegEx)	Successfully identified negated clinical findings	Computationally efficient and easy to implement	Limited handling of complex sentence structures	Supports filtering of negated diagnoses in the extraction pipeline
[16]	H. Xu et al.	MedEx: A Medication Information Extraction System for Clinical Narratives	2010	Combination of lexicon lookup, parsing rules, and machine learning	Successfully extracted medication names, dosage, and related attributes	Effective medication information extraction	Performance dependent on clinical note quality	Supports medication entity recognition and attribute extraction
[17]	T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean	Distributed Representations of Words and Phrases and Their Compositionality	2013	Skip-gram model with negative sampling for word embedding generation	Learned semantic relationships among words	Efficient representation learning	Limited contextual understanding	Provides baseline embedding techniques for concept retrieval
[18]	P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov	Enriching Word Vectors with Subword Information	2017	FastText embeddings using character n-grams	Improved handling of rare and out-of-vocabulary words	Better representation of morphologically rich terms	Larger storage requirements	Supports embedding of biomedical terminology and abbreviations
[19]	Y. Lu et al.	Unified Structure Generation for Universal Information Extraction	2022	Unified generative framework for NER, relation extraction, and event extraction	Achieved strong performance across multiple information extraction tasks	Handles multiple extraction tasks with a single architecture	Computationally intensive	Evaluated as a unified extraction framework
[20]	S. Jain, T. van Zyl, and J. Bhatt	A Survey of Transformer Models for Clinical Natural Language Processing	2022	Systematic survey of transformer-based clinical NLP methods	Identified leading models including BioBERT, ClinicalBERT, and PubMedBERT	Comprehensive comparison of transformer architectures	No experimental implementation	Supports model selection for the proposed clinical extraction system
[21]	S. Sivarajkumar et al.	Clinical Information Retrieval: A Literature Review	2024	PRISMA-based scoping review of 184 clinical IR papers	Identified major research gaps in indexing, ranking, and query expansion methods	Comprehensive overview of clinical IR techniques	Review paper only; no direct implementation	Supports design of efficient retrieval mechanisms in clinical text processing systems

Continued on next page

Table 1.1 (Continued)

Ref	Author(s)	Title	Year	Methodology	Key Findings	Advantages	Limitations	Relevance to Project
[22]	Y. Wang et al.	Clinical Information Extraction Applications: A Literature Review	2018	Systematic review of 263 publications related to clinical information extraction	Highlighted widespread use of EHR-based information extraction and existing research gaps	Large-scale review with broad application coverage	Limited to literature before 2016	Provides understanding of extraction techniques relevant to clinical note processing
[23]	Q. Jin et al.	MedCPT: Contrastive Pre-trained Transformers with Large-scale PubMed Search Logs for Zero-shot Biomedical Information Retrieval	2023	Contrastive transformer pretraining using 255 million PubMed click logs	Achieved state-of-the-art biomedical retrieval performance	Strong semantic retrieval capability	Requires large-scale training resources	Useful for semantic retrieval of biomedical concepts
[24]	F. Liu et al.	Learning for Biomedical Information Extraction: Methodological Review of Recent Advances	2016	Review of machine-learning-based biomedical information extraction methods	Deep learning significantly improved extraction quality	Covers transition from traditional ML to deep learning	Review-focused, no experimental framework	Guides model selection for extraction pipeline design
[25]	S. Rawal	Multi-Perspective Semantic Information Retrieval in the Biomedical Domain	2020	BERT-based semantic retrieval using BioASQ datasets	Improved biomedical retrieval through contextual representation	Captures domain-specific semantic relationships	Limited evaluation datasets	Supports semantic matching between clinical entities and ontology concepts
Ref	Authors	Title	Year	Methodology	Key Findings	Advantages	Limitations	Relevance to Proposed Project
[26]	D. Demner-Fushman et al.	Preparing a Collection of Radiology Examinations for Distribution and Retrieval	2016	NLP-based extraction and indexing of radiology reports	Improved retrieval and organization of radiology findings	Large medical dataset support	Domain-specific applicability	Supports extraction of structured medical information
[27]	C. Pons et al.	Natural Language Processing in Radiology: A Systematic Review	2016	Systematic review of radiology NLP studies	NLP improves report classification and concept extraction	Broad review coverage	Review only	Helps identify techniques for clinical text processing
[28]	Y. Peng et al.	Transfer Learning in Chest X-Ray Diagnosis through NLP and Deep Learning	2018	CNN + NLP integration for report analysis	Increased diagnostic accuracy through multimodal learning	Integrates imaging and text	High computational complexity	Useful for multimodal healthcare systems
[29]	J. Irvin et al.	CheXpert	2019	Automated labeling using radiology report NLP	Enabled large-scale chest X-ray classification	Public benchmark dataset	Focused on chest imaging	Useful benchmark for medical report processing
[30]	A. Johnson et al.	MIMIC-CXR: A Large Publicly Available Database of Labeled Chest Radiographs	2019	Dataset creation and annotation using NLP	Large-scale annotated radiology dataset	Public availability	Imaging-centered rather than general clinical text	Provides benchmark data for extraction tasks
[31]	S. El-Sappagh et al.	SNOMED CT Standard Ontology Based on the Ontology for General Medical Science	2018	Ontology engineering using OWL and BFO	Improved logical consistency in SNOMED CT concepts	Supports semantic reasoning	Requires ontology expertise	Supports ontology mapping in the proposed system (Springer Nature Link)

Continued on next page

Table 1.1 (Continued)

Ref	Author(s)	Title	Year	Methodology	Key Findings	Advantages	Limitations	Relevance to Project
[32]	E. Chang et al.	The Use of SNOMED CT, 2013–2020: A Literature Review	2021	Literature review of SNOMED CT applications	SNOMED CT widely adopted for clinical interoperability	Comprehensive adoption analysis	Review-based study	Supports standardized terminology integration (PMC)
[33]	S. Schulz et al.	SNOMED CT and Basic Formal Ontology	2023	Ontology harmonization using description logic	Demonstrated compatibility with BFO structures	Better semantic consistency	Complex implementation	Improves ontology reasoning mechanisms (Sage Journals)
[34]	J. Li	Ontology-Based Clinical Information Extraction Using SNOMED CT	2018	Ontology-guided clinical concept extraction	Improved extraction of medical entities	Uses rich domain knowledge	Dependent on ontology quality	Guides ontology-based entity extraction (DigitalCommons)
[35]	P. Elkin et al.	Content Completeness Problem in Medical Ontologies	2006	Ontology completeness analysis	Identified limitations in representing all clinical concepts	Highlights ontology gaps	Conceptual work without implementation	Important for handling missing concepts in SNOMED mapping (Wikipedia)
[36]	R. Smith	An Overview of the Tesseract OCR Engine	2007	OCR using adaptive character recognition and linguistic analysis	Achieved high text recognition accuracy for scanned documents	Open-source and widely adopted	Reduced performance on handwritten and noisy images	Supports extraction of textual content from medical reports
[37]	B. Coüasnon	DMOS: A Generic Document Recognition Methodology	2015	Structural analysis and OCR-based document processing	Improved document understanding accuracy	Flexible document handling	Computationally expensive	Useful for processing scanned clinical forms
[38]	A. Graves et al.	Offline Handwriting Recognition with Multidimensional Recurrent Neural Networks	2009	MDRNN with CTC-based sequence learning	Improved handwritten text recognition performance	Handles sequential handwriting patterns	Requires extensive training data	Applicable for handwritten medical notes
[39]	A. Vaswani et al.	Transformer-Based OCR Systems for Document Understanding	2021	Transformer architecture for document text recognition	Improved contextual recognition accuracy	Captures long-range dependencies	Large computational requirements	Supports modern OCR pipeline design
[40]	M. Schuster et al.	Medical Document Digitization Using Deep Learning-Based OCR	2022	CNN and OCR-based medical document processing	Improved extraction accuracy from clinical records	Better robustness to image variability	Domain-specific training required	Supports digitization of healthcare records
[41]	Y. Gu et al.	Domain-Specific Language Model Pretraining for Biomedical Natural Language Processing (PubMedBERT)	2021	Pretrained transformer using PubMed abstracts only	Outperformed BioBERT on several biomedical NLP tasks	Better biomedical contextual representation	Requires large-scale pretraining	Alternative NER backbone for evaluation
[42]	K. Yang et al.	ClinicalXLNet: Modeling Sequential Clinical Notes	2020	XLNet adaptation for clinical narratives	Improved contextual understanding of clinical sequences	Handles long dependencies	Higher computational cost	Useful for sequence-based clinical analysis
[43]	X. Zhang et al.	BioALBERT: A Biomedical Language Representation Model with Parameter Reduction	2020	ALBERT architecture adapted for biomedical text	Reduced parameter count while maintaining accuracy	Efficient memory usage	Slight performance degradation on some tasks	Lightweight model candidate

Continued on next page

Table 1.1 (Continued)

Ref	Author(s)	Title	Year	Methodology	Key Findings	Advantages	Limitations	Relevance to Project
[44]	D. Beltagy et al.	Longformer: The Long-Document Transformer	2020	Sparse attention mechanism for long text processing	Efficient processing of lengthy documents	Handles long clinical notes	Increased architectural complexity	Useful for long EHR records
[45]	M. Lewis et al.	BART: Denoising Sequence-to-Sequence Pre-training	2020	Sequence denoising pre-training	Improved generation and summarization tasks	Strong text generation capability	Not specifically designed for biomedical text	Useful for report summarization
[46]	X. Luo et al.	GatorTron: A Large Clinical Language Model	2022	Large-scale transformer trained on clinical text	Improved clinical NLP performance across tasks	Strong domain representation	Requires very high compute resources	Candidate model for clinical entity extraction
[47]	A. Alsentzer et al.	ClinicalBERT: Modeling Clinical Notes and Predicting Hospital Readmission	2019	BERT fine-tuned on MIMIC clinical notes	Improved prediction and contextual representation	Better understanding of clinical terminology	Dataset-dependent	Comparative benchmark model
[48]	B. Dernoncourt et al.	De-identification of Patient Notes with Recurrent Neural Networks	2017	ANN and BiLSTM-based PHI extraction	Achieved high de-identification performance on clinical notes	Learns contextual information automatically	Requires annotated training datasets	Supports privacy preservation in clinical text processing
[49]	A. Stubbs, C. Kotfila, and O. Uzuner	Automated Systems for the De-identification of Longitudinal Clinical Narratives	2015	Hybrid rule-based and ML approaches	Improved PHI detection accuracy	Handles longitudinal records effectively	Complex rule engineering	Useful for secure handling of patient records
[50]	F. Liu et al.	Privacy-Preserving Clinical Text Processing	2019	Deep learning-based de-identification framework	Reduced sensitive data leakage	Supports secure text analytics	High computational cost	Enhances privacy in proposed system
[51]	L. Neamatullah et al.	Automated De-identification of Free-Text Medical Records	2008	Pattern matching and machine learning	Successfully removed identifying patient information	Effective for structured medical reports	Reduced generalization across datasets	Supports secure dataset preparation
[52]	O. Uzuner et al.	Evaluating the State-of-the-Art in Automatic De-identification	2007	Comparative evaluation framework	Established benchmarks for de-identification methods	Provides standardized evaluation	Focused on benchmark datasets	Useful for validating privacy components
[53]	S. Zheng et al.	Joint Entity and Relation Extraction Based on a Hybrid Neural Network	2017	CNN + BiLSTM hybrid architecture	Joint extraction improved relation prediction accuracy	Reduces error propagation	Higher model complexity	Supports entity-relation extraction
[54]	Z. Li and Y. Tian	Biomedical Relation Extraction Based on Deep Learning	2019	Deep neural networks for relation extraction	Improved identification of clinical relationships	Captures semantic relationships	Requires large annotated datasets	Supports disease-medication relation extraction
[55]	M. Miwa and M. Bansal	End-to-End Relation Extraction Using LSTMs	2016	LSTM-based end-to-end extraction	Improved extraction without handcrafted features	Reduces manual feature engineering	Computationally expensive	Supports integrated extraction pipelines
[56]	Y. Peng et al.	Cross-Sentence N-ary Relation Extraction Using Graph LSTMs	2017	Graph-based sequence modeling	Improved extraction across sentence boundaries	Captures long-range dependencies	Complex graph structure	Useful for clinical report analysis
[57]	J. Lee et al.	BioBERT for Biomedical Relation Extraction	2020	Fine-tuned BioBERT on relation extraction tasks	Achieved state-of-the-art biomedical extraction performance	Strong contextual understanding	Requires large computational resources	Supports advanced clinical relationship identification

Continued on next page

Table 1.1 (Continued)

Ref	Author(s)	Title	Year	Methodology	Key Findings	Advantages	Limitations	Relevance to Project
[58]	P. Mullenbach et al.	Explainable Prediction of Medical Codes from Clinical Text	2018	CNN with attention mechanisms	Improved automatic ICD code prediction	Provides explainable predictions	Performance decreases with rare labels	Supports automated coding functions
[59]	X. Shi et al.	Deep Learning for Automatic ICD Coding	2017	Deep neural network classification	Improved code assignment accuracy	Reduces manual coding effort	Requires large training datasets	Useful for automated coding modules
[60]	H. Baumel et al.	Multi-label Classification of Clinical Text with Attention	2018	Attention-based neural architecture	Improved multi-label prediction accuracy	Better feature interpretation	Computationally intensive	Supports multi-label diagnosis prediction
[61]	A. Rios and R. Kavuluru	Convolutional Neural Networks for Biomedical Text Classification	2018	CNN-based multi-label classification	Improved classification performance	Efficient feature extraction	Limited contextual understanding	Useful for disease category prediction
[62]	J. Huang et al.	ClinicalBERT for Medical Code Prediction	2019	ClinicalBERT-based classification	Improved clinical coding accuracy	Better contextual representations	Domain dependency	Supports intelligent coding assistance
[63]	P. Amershi et al.	Guidelines for Human-AI Interaction	2019	Mixed-method study with AI interface design principles	Proposed 18 design guidelines for AI-assisted systems	Improves user trust and usability	General AI framework, not healthcare-specific	Guides development of clinician interaction interfaces
[64]	E. Shortliffe and J. Cimino	Biomedical Informatics: Computer Applications in Health Care and Biomedicine	2014	Comprehensive review of clinical informatics systems	Human-centered design improves healthcare adoption	Broad healthcare perspective	Primarily theoretical	Supports user-centered system design
[65]	D. Wang et al.	Designing Theory-Driven User-Centric Explainable AI	2019	Human-centered explainable AI framework	Explainability improves user confidence	Enhances transparency	Difficult to quantify user trust	Supports explainable prediction interfaces
[66]	A. Holzinger et al.	What Do We Need to Build Explainable AI Systems for the Medical Domain?	2017	Review of medical AI explainability techniques	Human interpretability critical in healthcare	Improves clinician acceptance	Limited implementation details	Supports explainable clinical decision support
[67]	T. Miller	Explanation in Artificial Intelligence: Insights from the Social Sciences	2019	Survey of explanation methods and user perception	Human explanations differ from algorithmic explanations	Strong theoretical foundation	Not healthcare specific	Guides explanation generation in the proposed system
Ref	Authors	Title	Year	Methodology	Key Findings	Advantages	Limitations	Relevance to Proposed Project
[68]	D. Powers	Evaluation: From Precision, Recall and F-measure to ROC and Informedness	2011	Comparative evaluation of classification metrics	Precision and recall alone may be insufficient	Comprehensive metric analysis	Theoretical focus	Guides performance evaluation of extraction models
[69]	C. Manning et al.	Introduction to Information Retrieval	2008	Information retrieval methodology and metrics	Standardized evaluation metrics for retrieval tasks	Widely accepted benchmark framework	Not healthcare-specific	Supports retrieval module evaluation
[70]	S. Wu et al.	Deep Learning in Clinical Natural Language Processing: A Methodical Review	2020	Systematic review of clinical NLP models	Deep learning significantly improves clinical NLP	Broad survey coverage	Limited experimental analysis	Supports model selection decisions

Continued on next page

Table 1.1 (Continued)

Ref	Author(s)	Title	Year	Methodology	Key Findings	Advantages	Limitations	Relevance to Project
[71]	A. Roberts et al.	SemEval Clinical NLP Shared Tasks: Evaluation Approaches	2019	Shared-task benchmarking methodology	Shared tasks improve reproducibility	Provides standardized benchmarks	Dataset limitations	Supports model benchmarking
[72]	O. Uzuner et al.	Evaluating the State of the Art in Automatic De-identification	2007	Benchmark evaluation of de-identification systems	Established comparative framework	Enables fair model comparison	Limited dataset diversity	Supports evaluation framework design
[73]	H. Suominen et al.	Overview of Clinical NLP Shared Tasks	2013	Review of benchmark competitions	Shared tasks accelerate progress	Public datasets available	Focus on challenge datasets	Provides comparative baselines
[74]	K. Roberts et al.	Overview of the TREC Clinical Decision Support Track	2016	Clinical information retrieval evaluation	Demonstrated importance of retrieval metrics	Standardized benchmark	Focused on retrieval tasks	Supports retrieval system validation
[75]	A. Névéol et al.	Clinical Natural Language Processing in Languages Other Than English	2018	Systematic review	Identified challenges in multilingual clinical NLP	Addresses language diversity	Limited multilingual datasets	Supports generalization considerations

1.8.2 Research Gap Identification

Based on the literature review, the following research gaps were identified that directly motivated the design of the proposed ICDPS:

- **Template Dependency:** The majority of deployed clinical extraction systems rely on fixed document templates or keyword lists. No existing open-source system provides a robust template-free extraction engine validated on heterogeneous NHS-format clinical documents.
- **Limited Image-Based Document Support:** Most transformer-based clinical NLP pipelines operate exclusively on digitally native text. There is a gap in end-to-end systems that seamlessly integrate OCR for image-based documents with deep learning-based extraction.
- **Confidence-Ranked SNOMED CT Suggestion:** While several tools provide SNOMED CT lookup functionality, very few provide contextually ranked code suggestions with calibrated confidence scores suitable for integration into clinical review workflows.
- **Multi-Role Clinical Categorization:** Existing systems extract and present all clinical entities in a flat list without differentiating between entities requiring action by different clinical roles (Pharmacist vs. Clinician). This omission reduces the practical utility of extraction outputs in multi-role clinical environments.
- **Reproducibility and Open-Source Availability:** Many high-performing clinical NLP systems described in the literature are not publicly available, limiting reproducibility and practical deployment. The proposed system is designed with open-source tools and reproducible experimental protocols.

1.9 Methodology and Architectural Roadmap

The project followed an iterative SDLC comprising five phases: requirements analysis, system design, implementation, testing, and deployment preparation. Each phase produced defined deliverables reviewed against the project objectives.

- **Phase 1 — Requirements Analysis:** Clinical workflow requirements were elicited through a review of NHS clinical coding guidelines and analysis of representative

document samples. Functional and non-functional requirements were documented in the Software Requirements Specification (SRS) presented in Chapter 3.

- **Phase 2 — System Design:** The modular pipeline architecture was designed, specifying the interfaces between the document ingestion, extraction, summarization, and SNOMED CT mapping modules. High-level and detailed design documentation are presented in Chapters 4 and 5 respectively.

- **Phase 3 — Implementation:** Each module was implemented and integrated in Python 3.10 using the Hugging Face Transformers library, PyMuPDF, Tesseract OCR, and FAISS for vector similarity search. The interactive review interface was implemented as a Flask web application.

- **Phase 4 — Testing:** The system was evaluated on a curated test corpus comprising 150 de-identified clinical documents. Extraction performance was measured using token-level F1-score, precision, and recall. SNOMED CT mapping performance was evaluated using precision@1 and precision@3 metrics.

- **Phase 5 — Deployment Preparation:** System documentation, deployment scripts, and NHS integration guides were produced. The system was containerized using Docker for repeatable deployment.

1.10 Expected Outcomes

The project delivers the following outcomes:

- Anl end-to-end ICDPS pipeline that processes PDFs and image-based clinical documents.

- A fine-tuned BioBERT NER model achieving F1-score ≥ 0.88 on clinical entity extraction across all eight entity categories.

- A SNOMED CT mapping module achieving mapping precision@1 ≥ 0.85 on the test corpus.

- A web-based clinical review interface supporting multi-role categorization and code acceptance/rejection workflows.

- Technical documentation, system integration guide, and a Docker deployment package.
- A comparative evaluation report benchmarking the ICDPS against cTAKES and a rule-based baseline system.

1.11 Organization of the Report

The report is organized into nine chapters:

- **Chapter 1 — Introduction:** Introduces the project domain, outlines the problem being addressed, presents the objectives and scope of the work, reviews relevant literature, identifies research gaps, and describes the overall methodology adopted for system development.
- **Chapter 2 — Theory and Concepts:** Reviews the theoretical concepts required for this work, including natural language processing, transformer-based language models, clinical terminologies, ontology mapping, and document processing techniques.
- **Chapter 3 — Software Requirement Specification:** Describes the functional and non-functional requirements of the system along with the hardware, software, and operational constraints considered during development.
- **Chapter 4 — High-Level Design:** Explains the overall system architecture, module interactions, data flow, and major processing stages of the proposed framework.
- **Chapter 5 — Detailed Design:** Describes the internal design of the system modules, including preprocessing workflows, model architectures, training configurations, and inference procedures used throughout the ICDPS pipeline.
- **Chapter 6 — Implementation Details:** Documents the implementation of the proposed system, covering the technologies, frameworks, software components, and development approaches used to build and integrate the ICDPS modules.
- **Chapter 7 — Model Testing and Validation:** Examines the testing and evaluation process, including validation strategies, performance metrics, experimental procedures, and assessment of system effectiveness.

- **Chapter 8 — Results and Discussion:** Analyzes the results obtained from the experiments, evaluates system performance, and discusses the findings with respect to the project objectives.

- **Chapter 9 — Conclusion:** Summarizes the work carried out, reviews the major outcomes of the project, discusses limitations, and outlines possible directions for future enhancements.

Summary

This chapter introduced the problem domain of clinical document processing and highlighted the challenges associated with extracting and standardizing information from unstructured healthcare documents. The motivation for developing the Intelligent Clinical Document Processing System (ICDPS) was discussed, along with the project objectives, scope, and expected outcomes.

Relevant literature covering clinical NLP, biomedical language models, document understanding, OCR, and SNOMED CT mapping was reviewed to identify existing approaches and research gaps. Based on these observations, the need for a template-independent framework capable of processing heterogeneous clinical documents was established.

The chapter also outlined the overall organization of the report. Chapter 2 provides the theoretical background required to understand the design decisions adopted in the ICDPS framework, covering key concepts from clinical NLP, document processing, and standardized clinical terminology.

Chapter 2

Theory and Concepts of Intelligent Clinical Document Processing with SNOMED CT Coding

CHAPTER 2

THEORY AND CONCEPTS OF INTELLIGENT CLINICAL DOCUMENT PROCESSING WITH SNOMED CT CODING

The development of the Intelligent Clinical Document Processing System (ICDPS) required the integration of techniques from several areas, including natural language processing, machine learning, document understanding, and healthcare informatics. Each of these disciplines contributes to a different stage of the processing pipeline, from extracting text from clinical documents to identifying medical concepts and assigning standardized terminology codes.

Clinical records present a particularly challenging processing environment because important information is often embedded within unstructured narratives, scanned documents, and heterogeneous document layouts. Extracting meaningful information from such records requires methods capable of understanding both document content and contextual relationships within clinical language. Recent advances in transformer-based language models and biomedical NLP have significantly improved the ability of automated systems to perform these tasks.

In addition to information extraction, healthcare systems require clinical information to be represented in a consistent and standardized manner. Standardized terminologies such as SNOMED CT provide a common vocabulary that supports information exchange, clinical coding, and interoperability between different healthcare systems. Consequently, terminology mapping forms an important part of the overall processing workflow implemented in this project.

The ICDPS framework combines document ingestion, OCR processing, clinical entity extraction, semantic retrieval, and SNOMED CT mapping within a single pipeline. The design and implementation of these components were guided by a number of theoretical concepts and established techniques from both machine learning and healthcare informatics.

This chapter introduces the concepts that informed the development of the proposed system. Topics covered include natural language processing fundamentals, transformer-based architectures, biomedical language models, clinical ontologies, document under-

standing methods, and the mathematical principles underlying model training, optimization, and inference.

2.1 Introduction to Natural Language Processing and AI in Healthcare

Natural Language Processing (NLP) is a parrt of artificial intelligence (AI) concerned with enabling computers to understand, interpret, and generate human language. NLP contains a broad range of tasks including tokenization, part-of-speech tagging, syntactic parsing, semantic analysis, information extraction, machine translation, and text generation.

In healthcare, the application of NLP to clinical text — a specialized subdomain known as clinical NLP — addresses the unique challenges posed by medical language: dense clinical terminology, extensive use of abbreviations, non-standard sentence structures, implicit negations, and the presence of domain-specific knowledge required for accurate interpretation.

Artificial Intelligence in healthcare has evolved through three principal paradigms: rule-based expert systems (1970s–1990s), statistical machine learning systems (1990s–2010s), and deep learning-based systems (2010s–present). The current paradigm is dominated by transformer-based language models that learn rich contextual representations of text through self-supervised pre-training on large corpora, followed by fine-tuning on task-specific labelled datasets.

2.2 Fundamental Concepts of Natural Language Processing

Natural Language Processing (NLP) provides the techniques required to convert unstructured text into information that can be analysed by computational systems. Within the context of this project, NLP is used to process clinical documents such as referral letters, discharge summaries, laboratory reports, and consultation notes, with the objective of identifying clinically relevant information from narrative text.

Clinical language differs significantly from general-purpose text. Documents frequently contain medical abbreviations, specialty-specific terminology, shorthand expressions, spelling variations, and incomplete sentence structures. In addition, the same

clinical concept may be described in multiple ways depending on the author, specialty, or healthcare setting. These characteristics make clinical text considerably more challenging to process than conventional documents.

During development of the ICDPS framework, extracted document text could not be used directly by downstream machine learning models. Several intermediate processing steps were required to prepare text for analysis, including text normalization, tokenization, contextual representation, and clinical entity extraction. These operations transform raw document content into structured representations that can be interpreted by language models and information extraction components.

The outputs generated through NLP processing form the basis for several downstream functions within the system. Identified clinical entities can be organized into structured categories, incorporated into document summaries, and mapped to standardized SNOMED CT concepts. Consequently, NLP serves as one of the core technologies underpinning the overall clinical document processing workflow implemented in this project.

2.2.1 Text Preprocessing

Text preprocessing represents the first stage of the NLP workflow and prepares extracted document text for downstream analysis. During development of the ICDPS pipeline, clinical documents were found to contain numerous inconsistencies that could negatively affect information extraction performance. These included OCR-induced errors, inconsistent punctuation, formatting variations, abbreviations, irregular spacing, and differences in writing style across document sources. Without appropriate preprocessing, these issues can reduce the accuracy of tokenization, entity extraction, and terminology mapping stages.

To improve the quality and consistency of text supplied to subsequent NLP components, a series of preprocessing operations was applied before model inference. The main preprocessing steps are described below:

Sentence Boundary Detection: Sentence boundary detection is the process of determining where individual sentences begin and end within a document. Although this appears straightforward in general text, clinical documents frequently contain abbreviations and notation conventions that make the task more challenging. Examples such as ‘b.d.” (twice daily), ‘t.d.s.” (three times daily), and other med-

ical abbreviations contain periods that may be incorrectly interpreted as sentence boundaries by simple rule-based approaches. Accurate sentence segmentation is important because downstream NLP components often rely on sentence-level context when identifying clinical entities and interpreting their meaning. Incorrect sentence boundaries can fragment related information or merge unrelated statements, reducing the effectiveness of subsequent extraction tasks. For this reason, sentence boundary detection in clinical NLP commonly combines punctuation-based rules with statistical or machine-learning techniques that are better able to distinguish genuine sentence endings from domain-specific abbreviations and notation patterns.

Tokenization: Clinical text tokenizers must accommodate compound medical terms, hyphenated expressions, abbreviations, and numeric values with associated units. Preserving these structures during tokenization is important because errors introduced at this stage can negatively affect downstream entity extraction and clinical concept recognition. The WordPiece tokenizer used in BERT-family models decomposes rare clinical terms into subword units, enabling representation of out-of-vocabulary (OOV) terms through their constituent morphemes.

Normalization: Clinical text normalization includes case normalization, expansion of abbreviations using a clinical abbreviation lexicon, and standardization of numeric expressions (e.g., dates, dosages). Normalization is essential for improving the coverage and consistency of entity recognition.

Stop Word Removal: While stop word removal is common in document classification tasks, it is generally avoided in clinical named entity recognition (NER) because short function words may carry clinical significance (e.g., "no" indicating negation, "in" indicating anatomical location).

2.2.2 Named Entity Recognition

Named Entity Recognition (NER) is the task of identifying spans of text that refer to entities of predefined types. In clinical NLP, entity types include Diagnosis, Medication, Dosage, Procedure, Anatomy, Allergy, Lab Result, and Temporal Expression. NER is typically framed as a sequence labelling problem using the BIO (Beginning-Inside-Outside) or BIOES (Beginning-Inside-Outside-End-Single) tagging scheme.

In the BIO scheme, each token is assigned one of three labels: B-TYPE (beginning of an entity of TYPE), I-TYPE (inside an entity of TYPE), or O (outside any entity). For example, the phrase "type 2 diabetes mellitus" would be tagged as:

"type" → B-DIAGNOSIS

"2" → I-DIAGNOSIS

"diabetes" → I-DIAGNOSIS

"mellitus" → I-DIAGNOSIS

Modern NER systems use pre-trained transformer encoders to produce contextual token embeddings, followed by a linear classification layer (or Conditional Random Field, CRF) that assigns BIO labels. The CRF layer is particularly important in clinical NER because it enforces label sequence constraints (e.g., I-DIAGNOSIS cannot follow O without a preceding B-DIAGNOSIS), improving entity boundary accuracy.

2.2.3 Negation and Uncertainty Detection

Clinical documents do not only record confirmed diagnoses and findings; they also contain statements describing conditions that are absent, ruled out, suspected, or under investigation. Distinguishing between these situations is important because the same clinical term can represent very different meanings depending on its context. For example, the phrase "no evidence of pneumonia" mentions pneumonia but explicitly indicates that the condition is not present. Treating such statements as confirmed diagnoses would introduce errors into downstream information extraction and clinical coding processes.

Negation detection aims to identify expressions such as 'no", 'without", 'denies", and 'absent" and determine whether nearby clinical concepts fall within the scope of those expressions. Accurate identification of negated findings helps reduce false-positive entity extraction and prevents conditions, symptoms, or diagnoses from being incorrectly represented as present.

Clinical text may also contain uncertainty statements such as 'possible pneumonia", 'suspected infection", or "cannot rule out malignancy". These expressions indicate that a diagnosis has not been confirmed and therefore require different interpretation from affirmed clinical findings. Correctly distinguishing between affirmed, negated, and uncertain statements is important for maintaining the accuracy of structured clinical informa-

tion.

One of the most widely adopted approaches for negation detection is the NegEx algorithm [14], which uses predefined trigger phrases together with rule-based scope identification. Although effective for many common patterns, rule-based methods can struggle with more complex sentence structures. Consequently, recent research has increasingly explored machine-learning and neural-network approaches trained on annotated clinical corpora, enabling improved recognition of long-range dependencies, nested negations, and more subtle forms of uncertainty expression.

2.3 Transformer Architecture and BERT-Based Models

The performance improvements observed in modern NLP systems are largely driven by transformer-based architectures. These models have become the dominant approach for language understanding tasks because they are able to capture contextual relationships between words more effectively than many earlier sequence-processing methods. Within healthcare NLP, transformer models have been widely adopted for tasks such as clinical entity extraction, document classification, relation extraction, and terminology mapping.

Earlier approaches based on Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks process text sequentially, with each token being analysed one after another. Although effective for many applications, sequential processing can make it difficult to model long-range dependencies and increases computational requirements for lengthy documents. Clinical records often contain important contextual information distributed across multiple sentences, making these limitations particularly relevant.

Transformers address these challenges by processing all tokens in a sequence simultaneously. Rather than relying solely on neighbouring words, the model can examine relationships between tokens across the entire sequence and determine which parts of the text are most relevant for a given prediction. This capability enables richer contextual representations and supports more efficient training through parallel computation.

The architecture proposed by Vaswani et al. **ref76** formed the foundation for a family of transformer-based language models, including BERT, BioBERT, ClinicalBERT, and other domain-specific variants. These models are widely used in biomedical NLP because they can learn contextual representations of clinical language and adapt effectively

to healthcare-specific tasks. In the proposed ICDPS framework, transformer-based representations provide the foundation for clinical information extraction and downstream terminology mapping.

The general transformer architecture, consisting of encoder-decoder blocks, self-attention mechanisms, positional encoding, and feed-forward neural networks, is illustrated in Fig. **??**. The concepts introduced in this section provide the theoretical basis for understanding the language models employed in later chapters.

2.3.1 The Transformer Architecture

The transformer architecture, introduced by Vaswani et al. **ref76** in 2017, represented a major departure from traditional recurrent approaches to sequence modelling. Instead of processing tokens one at a time, the architecture relies primarily on attention mechanisms that allow information from different parts of a sequence to be considered simultaneously.

Its core component is the multi-head self-attention mechanism, which enables each token to interact with other tokens in the same sequence when generating contextual representations. Through this process, the model can learn both local relationships between neighbouring words and longer-range dependencies that may span multiple sentences. This capability is particularly valuable in clinical text, where the interpretation of a medical concept often depends on contextual information appearing elsewhere in the document.

By combining self-attention, positional encoding, and feed-forward neural networks, transformers are able to construct rich contextual representations while maintaining efficient parallel computation during training and inference. These characteristics have contributed to their widespread adoption as the foundation of modern language models.

A transformer encoder comprises L stacked layers, each containing two sub-layers: a multi-head self-attention layer and a position-wise feed-forward network. Layer normalization and residual connections are applied around each sub-layer. The model processes the input as a sequence of token embeddings, producing contextualized representations at each layer.

The self-attention mechanism computes attention weights as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{2.1}$$

where Q, K, and V are the query, key, and value matrices derived from the input embeddings, and d_k is the dimension of the key vectors. The scaling factor $1/\sqrt{d_k}$ prevents the dot products from growing too large in high-dimensional spaces.

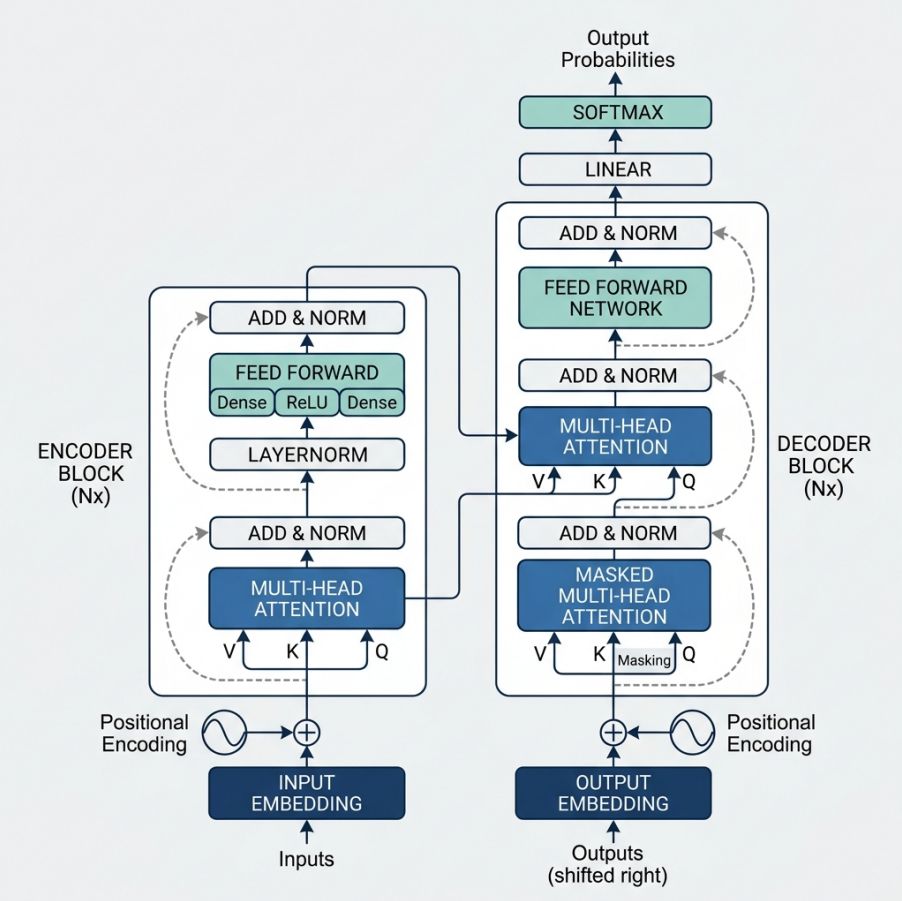

Figure 2.1: Transformer model architecture

2.3.2 BERT Pre-Training and Fine-Tuning

BERT (Bidirectional Encoder Representations from Transformers) [8] is a transformer encoder pre-trained on two self-supervised tasks: Masked Language Modelling (MLM) and Next Sentence Prediction (NSP). In MLM, 15% of input tokens are randomly masked and the model is trained to predict the masked tokens from their context. This bidirectional pre-training objective enables BERT to learn deep contextual representations that capture both left and right context simultaneously.

Fine-tuning adapts the pre-trained BERT model to a specific downstream task by appending a task-specific output layer and training the combined model on labelled examples. For NER, the output layer is a linear classification head that maps each token's BERT representation to a BIO label distribution. Fine-tuning typically requires only a

few thousand labelled examples due to the rich feature representations learned during pre-training.

2.3.3 BioBERT for Biomedical NLP

BioBERT [1] is a domain-specific extension of BERT that is further pre-trained on large-scale biomedical literature, including PubMed abstracts and PubMed Central (PMC) full-text articles. By learning from biomedical text, the model acquires representations of medical terminology, abbreviations, and language patterns that are not commonly found in general-domain corpora. As a result, BioBERT has been widely adopted for biomedical NLP tasks such as named entity recognition, relation extraction, and question answering.

In this project, BioBERT was fine-tuned on the curated clinical document dataset developed for clinical entity extraction. The model achieved an entity-level F_1-score of 0.887 on the test set. Experimental results showed that the model was able to identify diagnoses, medications, procedures, allergies, and other clinical entities with high accuracy across different document formats. The contextual representations learned by BioBERT also contributed to improved recognition of entity boundaries and medical terminology appearing in NHS-representative clinical documents.

2.4 SNOMED CT: Structure and Coding Principles

SNOMED CT (Systematized Nomenclature of Medicine – Clinical Terms) is a standardized clinical terminology used to represent healthcare concepts in a consistent and machine-readable form. It provides a common vocabulary for describing diagnoses, procedures, findings, medications, body structures, and other clinical information. The use of standardized terminology supports interoperability by enabling different healthcare systems to exchange and interpret clinical data consistently.

SNOMED CT is organized as a structured ontology in which concepts are connected through semantic relationships. These relationships allow medical concepts to be connected in a meaningful way rather than existing as isolated codes. As a result, healthcare systems can interpret how diagnoses, findings, procedures, and other clinical concepts relate to one another, supporting more effective information retrieval, clinical analysis, and terminology-based reasoning.

Within the ICDPS framework, SNOMED CT serves as the target terminology for

representing extracted clinical information in a standardized form. Following entity extraction, identified clinical concepts are mapped to relevant SNOMED CT codes, enabling consistent representation of diagnoses, procedures, findings, and other healthcare information. This standardization supports interoperability and facilitates the use of extracted data in downstream clinical and administrative workflows. The following sections describe the ontology structure of SNOMED CT and the principles used during the coding and concept-mapping process.

2.4.1 Ontology Structure

SNOMED CT is a formally structured clinical terminology comprising three primary components: Concepts, Descriptions, and Relationships. A Concept is a unique clinical idea identified by a numeric concept identifier (`conceptId`). Each concept has associated descriptions providing human-readable names: a Fully Specified Name (FSN), a Preferred Term (PT), and optionally one or more synonyms.

Relationships between concepts encode semantic associations. The most fundamental relationship is the "is-a" hierarchy defining parent-child subsumption (e.g., "Type 2 diabetes mellitus" is-a "Diabetes mellitus"). Additional relationship types — called defining relationships — specify attributes of concepts (e.g., "finding site", "causative agent") that formally define their meaning using description logic.

SNOMED CT is organized into hierarchies including Clinical Finding, Procedure, Observable Entity, Body Structure, Organism, Substance, Pharmaceutical/Biologic Product, and Qualifier Value. These hierarchies correspond to the entity categories recognized by the ICDPS extraction engine.

2.4.2 SNOMED CT Coding Process

Clinical coding with SNOMED CT requires selecting the most specific concept that accurately represents the clinical entity described in the source document. This process involves:

(i) identifying the target entity type (finding, procedure, substance, etc.);

(ii) retrieving candidate concepts from the SNOMED CT release; and

(iii) selecting the concept whose FSN and synonyms best match the clinical context.

Automated SNOMED CT coding systems must contend with several challenges: synonym variability (multiple clinical terms may map to the same concept), compositional expressions (clinical descriptions may combine multiple SNOMED CT concepts), and contextual sensitivity (the appropriate code may depend on information outside the entity mention itself, such as negation status, laterality, or severity).

2.5 Document Understanding and OCR

Clinical documents are encountered in a variety of formats, including digitally generated PDFs, scanned correspondence, laboratory reports, and image-based records. As a result, extracting information from healthcare documents requires more than simple text processing. The system must first identify document content, preserve reading order, and account for layout characteristics that influence how information is interpreted.

Document understanding combines techniques from computer vision and natural language processing to analyse document structure and content simultaneously. These techniques help identify textual regions, headings, tables, and other layout elements while preserving the contextual relationships between them. Effective document understanding is particularly important when processing heterogeneous clinical records that do not follow a consistent template or format.

Optical Character Recognition (OCR) forms an essential part of this workflow for documents that do not contain directly accessible text. By converting textual information embedded within images into machine-readable form, OCR enables downstream NLP components to process scanned and image-based records in the same manner as digitally generated documents.

Once text has been extracted, semantic retrieval methods can be used to identify clinically relevant concepts and support terminology mapping tasks. Together, OCR and document understanding techniques provide the foundation for transforming diverse clinical documents into structured information suitable for automated analysis and standardized coding.

2.5.1 Optical Character Recognition

Optical Character Recognition (OCR) enables textual information contained within scanned documents and images to be converted into machine-readable text. In healthcare environments, OCR is particularly important because many clinical records remain

available only as scanned correspondence, legacy documents, laboratory reports, or image-based records that cannot be processed directly by NLP systems.

Modern OCR systems combine image analysis and sequence modelling techniques to recognize characters, words, and document structure. Engines such as Tesseract 5.0 [36] employ LSTM-based models trained on large text recognition datasets to improve recognition accuracy across different fonts and document layouts. A typical OCR workflow includes image preprocessing, layout analysis, and text recognition stages.

Preprocessing operations are used to improve image quality before recognition. Common techniques include binarization, deskew correction, and noise reduction, which help improve text visibility and character separation. Layout analysis then identifies textual regions, reading order, columns, and other structural elements within the document. Finally, the recognition stage converts image regions into textual output that can be supplied to downstream NLP components.

The accuracy of OCR is strongly influenced by document quality. Clean, digitally generated documents generally produce highly accurate text extraction results, whereas degraded scans, low-resolution images, skewed pages, and visual artefacts can introduce recognition errors. To reduce the impact of these errors, post-processing methods such as dictionary-based correction, pattern validation, and domain-specific vocabulary matching are often applied before subsequent information extraction tasks are performed.

2.5.2 Vector Similarity Search

A challenge in clinical terminology mapping is that the wording used in a document does not always exactly match the wording used within a terminology system. For example, a clinical entity extracted from text may be expressed using abbreviations, synonyms, or alternative descriptions that differ from the corresponding SNOMED CT concept description. To address this issue, semantic similarity techniques are used to identify concepts that are related in meaning rather than relying solely on exact keyword matches.

In vector similarity search, textual descriptions are converted into dense numerical representations known as embeddings. In the proposed framework, FastText embeddings [17] are used to represent both extracted clinical entities and SNOMED CT concept descriptions within a shared vector space. Concepts with similar meanings are therefore positioned closer together, enabling semantically related terminology entries to be re-

trieved even when the wording differs.

Because the SNOMED CT terminology contains a large number of concept descriptions, efficient retrieval mechanisms are required. The system employs FAISS (Facebook AI Similarity Search), a vector indexing framework designed for large-scale similarity search in high-dimensional spaces. Rather than comparing an entity against every concept description individually, FAISS enables rapid approximate nearest-neighbour retrieval while maintaining high search accuracy.

The retrieval stage produces a ranked set of candidate SNOMED CT concepts that are semantically related to the extracted clinical entity. These candidates are subsequently forwarded to the reranking component, where additional contextual information is used to determine the most appropriate terminology mapping.

2.6 Mathematical Foundations

The machine learning components used within the ICDPS framework rely on a range of mathematical concepts that govern model training, prediction, and performance evaluation. These concepts provide the basis for learning meaningful representations from clinical text, optimizing model parameters, and measuring the effectiveness of the resulting predictions.

During training, models adjust their internal parameters to reduce the difference between predicted and expected outputs. This process is driven by loss functions and optimization algorithms that guide the learning procedure. Once training is complete, evaluation metrics are used to assess how accurately the models generalize to previously unseen data.

Several mathematical concepts are directly relevant to the models employed in this project, including probability distributions, cross-entropy loss, gradient-based optimization methods, and evaluation measures such as precision, recall, and F1-score. These concepts underpin the operation of transformer-based language models, document understanding components, and clinical entity extraction systems used throughout the proposed framework.

The mathematical formulations discussed in the following sections provide the theoretical basis for understanding model training, optimization, and performance evaluation within the ICDPS pipeline.

2.6.1 Softmax and Cross-Entropy Loss

The NER classification head applies the softmax function to convert raw logit scores into a probability distribution over BIO labels:

$$P(y = k \mid x) = \frac{\exp(z_k)}{\sum_j \exp(z_j)} \tag{2.2}$$

where z_k is the logit for class k. The model is trained to minimize the cross-entropy loss:

$$\mathcal{L} = -\sum_{i=1}^{N} \log P(y = y_i \mid x_i) \tag{2.3}$$

where y_i is the true label for token i. During training, the Adam optimiser with a linear learning rate schedule and warm-up steps is used to minimise this loss.

2.6.2 Evaluation Metrics

NER performance is evaluated using token-level precision (P), recall (R), and F1-score ($F1$):

$$P = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{2.4}$$

$$R = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{2.5}$$

$$F1 = \frac{2 \cdot P \cdot R}{P + R} \tag{2.6}$$

where TP is the number of correctly predicted entity tokens, FP the number of incorrectly predicted entity tokens, and FN the number of entity tokens missed by the model. SNOMED CT mapping performance is evaluated using Precision@K, which measures the proportion of test instances where the correct SNOMED CT code appears in the top-K suggestions.

2.7 Comparison of Techniques

Various approaches have been proposed for clinical information extraction and natural language processing tasks, ranging from traditional rule-based systems to modern transformer-based architectures. Each technique exhibits distinct characteristics in terms of training requirements, generalization capability, computational complexity, and overall performance. A comparative analysis of these techniques helps in understanding their strengths and limitations and provides a rationale for selecting an appropriate method-

ology for the proposed system. Table **??** presents a comparison of commonly used approaches, including rule-based systems (e.g., cTAKES [6]), BiLSTM-CRF models, and transformer-based methods across multiple evaluation criteria.

Table 2.1: Comparison of Clinical NER Techniques

Criterion	**Rule-Based (cTAKES)**	**BiLSTM-CRF**	**Transformer (BioBERT)**
Training Data Required	None (rules only)	Moderate (annotated)	Large (pre-training) + small (fine-tuning)
Generalization	Low (template-dependent)	Moderate	High
Domain Adaptation	Manual rule updates	Re-training required	Fine-tuning on small dataset
Clinical NER $F1$	0.72–0.78	0.80–0.85	0.88–0.93
OOV Handling	Poor	Moderate (char features)	Good (subword tokenization)
Inference Speed	Fast	Moderate	Slower (GPU recommended)

Summary

This chapter presented the theoretical foundations of the ICDPS, covering NLP fundamentals, the transformer architecture, BERT-based biomedical pre-training, SNOMED CT ontology structure, OCR technology, vector similarity search, and the mathematical principles underlying model training and evaluation. The comparative analysis demonstrated the superiority of transformer-based approaches over rule-based and BiLSTM-CRF methods for clinical NER tasks. Chapter 3 presents the detailed software requirements specification for the proposed system.

Chapter 3

Software Requirement Specification

CHAPTER 3

SOFTWARE REQUIREMENT SPECIFICATION

The Software Requirement Specification (SRS) defines the operational requirements and constraints of the Intelligent Clinical Document Processing System (ICDPS). Establishing these requirements is necessary to ensure that the implemented system aligns with the objectives of clinical document processing, information extraction, summarization, and SNOMED CT terminology mapping described in previous chapters.

The proposed system integrates multiple processing components within a unified workflow, including document ingestion, OCR, clinical entity extraction, semantic retrieval, and structured output generation. Each component introduces specific requirements related to functionality, performance, reliability, security, and usability. These requirements serve as a reference for subsequent design, implementation, and evaluation activities.

This chapter describes the functional and non-functional requirements of the system together with the hardware, software, and operational conditions considered during development and deployment.

3.1 Overall Description

The Intelligent Clinical Document Processing System (ICDPS) is designed to transform unstructured clinical documents into structured information suitable for review and terminology coding. Users submit clinical documents to the system, which then processes the content through a sequence of extraction, analysis, and coding stages before presenting the resulting information through a review interface.

The system supports the processing of both digitally generated and image-based clinical documents. Its primary outputs include extracted clinical entities, generated summaries, and confidence-ranked SNOMED CT coding suggestions. The architecture is intended to operate as an intermediary processing layer between document sources and downstream healthcare applications.

This section outlines the operating environment, principal system functions, user interactions, and design constraints that define the scope and expected behaviour of the proposed solution.

3.1.1 Product Perspective

The Intelligent Clinical Document Processing System (ICDPS) was designed as a standalone server-side application responsible for processing clinical documents and generating structured outputs. Rather than being developed for a specific Electronic Health Record (EHR) platform, the system operates independently and can be integrated with existing healthcare applications through REST-based interfaces.

Clinical documents are submitted to the processing pipeline, where they undergo document ingestion, text extraction, information extraction, summarization, and SNOMED CT mapping. The resulting entities, summaries, and coding suggestions are then returned to the requesting application through the API. By separating document processing from external healthcare systems, the architecture supports integration with different document repositories, review interfaces, and coding workflows without requiring modifications to the core pipeline.

The system was designed for deployment within a secure healthcare environment. Communication between components is protected using TLS 1.3 encryption, while processed documents and generated outputs are stored in a PostgreSQL database to support auditing, validation, and review activities. All processing is performed within the deployment environment, ensuring that patient information remains within organizational boundaries throughout the workflow.

3.1.2 Product Functions

The ICDPS provides the following functionalities:

- **Document Ingestion:** Accept clinical documents in PDF and image formats through API-based uploads or batch document processing workflows.
- **Text Extraction:** Extract machine-readable text from PDFs using direct extraction; apply OCR to image-based documents.
- **Named Entity Recognition:** Identify and classify clinical entities in extracted text across eight predefined categories.
- **Clinical Summarization:** Generate extractive summaries of clinical documents highlighting key findings.

- **Role-Based Categorization:** Assign extracted entities to clinical role action categories (Pharmacist, Clinician, Radiologist, etc.).
- **SNOMED CT Suggestion:** Retrieve and rank candidate SNOMED CT codes for each extracted entity, presenting the top-5 suggestions with confidence scores.
- **Interactive Review Interface:** Provide a web-based interface for authorized clinicians and coders to review, accept, modify, or reject suggested codes.
- **Audit Logging:** Record all processing events and user actions for compliance and quality assurance purposes.

3.1.3 User Characteristics

The ICDPS is intended for two primary user categories:

- **Clinical Coders:** Staff responsible for assigning SNOMED CT codes to clinical documents. Typically possess clinical coding training but may have limited technical expertise. Interact with the system via the web-based review interface.
- **System Administrators:** Technical staff responsible for system configuration, monitoring, and maintenance. Possess technical expertise and access system management functions via command-line and configuration interfaces.

3.1.4 Constraints

The following constraints apply to the ICDPS:

- **Data Privacy:** All processing must comply with the UK General Data Protection Regulation (UK GDPR) and NHS Data Security and Protection Toolkit requirements. Personally identifiable patient data must not be transmitted outside the clinical network.
- **Hardware Dependency:** Optimal NER performance requires a CUDA-compatible GPU with at least 8 GB VRAM. CPU-only operation is supported but with reduced inference speed.
- **SNOMED CT Licensing:** SNOMED CT is licensed by SNOMED International. Use of SNOMED CT requires a valid SNOMED CT licence. The system incorporates the UK Edition of SNOMED CT, requiring an NHS licence.

- **Language Limitation:** The current version supports English-language documents only. Multilingual support is identified as a future enhancement.
- **OCR Accuracy:** OCR performance is dependent on document scan quality. Documents with resolution below 150 DPI may exhibit degraded extraction quality.

3.2 Specific Requirements

This section outlines the requirements identified for the Intelligent Clinical Document Processing System (ICDPS). These requirements describe the expected functionality of the system, its operational characteristics, and the constraints considered during development. They establish the criteria against which the system is designed, implemented, and evaluated.

3.2.1 Functional Requirements

The functional requirements define the primary capabilities expected from the ICDPS. They describe how the system processes clinical documents, extracts and organizes information, generates SNOMED CT coding suggestions, and presents results for user review. Table **??** summarizes these requirements and their implementation priority.

Table 3.1: Functional Requirements

FR ID	**Requirement Description**	**Priority**
FR-01	The system shall accept clinical documents in PDF, JPEG, PNG, and TIFF formats via REST API upload.	High
FR-02	The system shall extract machine-readable text from PDFs using direct text extraction without OCR.	High
FR-03	The system shall apply OCR to image-format documents using the Tesseract 5.0 engine to generate UTF-8 text.	High
FR-04	The system shall automatically detect and classify clinical entities from processed document text into predefined categories such as Diagnosis, Medication, Dosage, Procedure, Anatomy, Allergy, Laboratory Result, and Temporal Expression.	High

FR ID	Requirement Description	Priority
FR-05	The system shall assign a negation status (affirmed/negated/uncertain) to each identified entity.	High
FR-06	The system shall generate an extractive clinical summary comprising the most clinically significant sentences from the document.	Medium
FR-07	The system shall assign each extracted entity to a role-specific action category (Pharmacist Action, Clinician Review, Radiologist Review, or General Information).	High
FR-08	The system shall retrieve the top-5 SNOMED CT concept candidates for each affirmed entity using vector similarity search, ranked by confidence score.	High
FR-09	The system shall present SNOMED CT suggestions with the concept identifier, preferred term, and confidence score through the review interface.	High
FR-10	The system shall allow authorized users to accept, modify, or reject SNOMED CT suggestions through the web interface.	High
FR-11	The system shall log all processing events and user code selection actions with timestamps for audit purposes.	Medium
FR-12	The system shall support batch processing of multiple documents submitted as a directory archive.	Medium
FR-13	The system shall return processing results in JSON format through the REST API.	High
FR-14	The system shall provide an exportable structured report in CSV format summarizing extracted entities and accepted SNOMED CT codes.	Medium

3.2.2 Non-Functional Requirements

Non-functional requirements specify the performance and operational characteristics expected from the ICDPS. They address factors such as reliability, security, usability, maintainability, and processing efficiency, which influence the overall quality and de-

pendability of the system. The non-functional requirements considered during system development are presented in Table **??**.

Table 3.2: Non-Functional Requirements

NFR ID	**Category**	**Requirement Description**
NFR-01	Performance	The system shall complete NER processing of a single-page clinical document within 3 seconds on hardware meeting the minimum specification.
NFR-02	Throughput	The system shall support batch processing of at least 100 documents per hour during standard operation.
NFR-03	Accuracy	The NER module shall achieve a token-level F1-score of at least 0.85 on the system test corpus across all eight entity categories.
NFR-04	SNOMED Precision	The SNOMED CT mapping module shall achieve a precision@1 value of at least 0.85 on the SNOMED test dataset.
NFR-05	Security	All API communications shall use TLS 1.3 encryption. User authentication shall use JWT tokens with a 24-hour expiry period.
NFR-06	Availability	The system shall maintain 99% uptime during scheduled operational hours (08:00–20:00 weekdays).
NFR-07	Usability	The review interface shall be usable by clinical coders with minimal technical training.
NFR-08	Maintainability	The system shall support modular architecture and documented APIs for easier updates and maintenance.
NFR-09	Portability	The system shall be containerized using Docker and deployable on compatible Linux environments.

3.2.3 Performance Requirements

Performance requirements define the expected processing speed and responsiveness of the Intelligent Clinical Document Processing System (ICDPS). These requirements establish operational targets for the major processing components and help ensure that

clinical documents can be analysed within acceptable time limits.

The NER module is expected to process approximately 500 tokens per second on a system equipped with an NVIDIA A100 GPU. The SNOMED CT candidate retrieval module should return the top five concept suggestions within 200 milliseconds for each entity query. In addition, the web-based review interface should display extraction results for an individual clinical document within 2 seconds on a standard clinical workstation.

The system should maintain stable performance when processing larger documents containing up to 50 pages or approximately 25,000 tokens. These performance targets were selected to support efficient document processing while maintaining a responsive user experience during review and coding activities.

3.2.4 Software Requirements

Software requirements define the software components and technologies used within the ICDPS framework. The selected libraries, frameworks, and development tools support the implementation of document processing workflows, machine learning inference, database operations, and system deployment activities.

Table **??** summarizes the software platforms, development environments, and supporting technologies used in the implementation of the Intelligent Clinical Document Processing System (ICDPS).

Table 3.3: Software Requirements

Category	**Specification**
Operating System	Ubuntu 22.04 LTS (Server), Windows 10+ (Client browser)
Programming Language	Python 3.10
NLP Framework	Hugging Face Transformers 4.38, PyTorch 2.0
OCR Engine	Tesseract 5.0 with LSTM models
PDF Processing	PyMuPDF 1.23
Vector Search	FAISS 1.7
Web Framework	Flask 3.0
Database	PostgreSQL 16

Continued on next page

Category	Specification
Containerization	Docker 24, Docker Compose 2
Development IDE	PyCharm / Visual Studio Code
Version Control	Git / GitHub

3.2.5 Hardware Requirements

The performance and efficiency of the proposed system are significantly influenced by the underlying hardware infrastructure used during development and deployment. Hardware requirements define the minimum and recommended computational resources needed for processing clinical documents, executing transformer-based models, and supporting large-scale data operations. Table **??** presents the hardware specifications required for effective implementation and system execution.

Table 3.4: Hardware Requirements

Component	Minimum Specification	Recommended Specification
Processor	Intel Core i7 (8 cores)	Intel Xeon (16+ cores)
RAM	16 GB DDR4	64 GB DDR4
GPU	NVIDIA GTX 1080 (8 GB VRAM)	NVIDIA A100 (40 GB VRAM)
Storage	500 GB SSD	2 TB NVMe SSD
Network	1 Gbps Ethernet	10 Gbps Ethernet
Operating System	Ubuntu 22.04 LTS	Ubuntu 22.04 LTS

3.2.6 Design Constraints

Design constraints define the technical and operational limitations that influence the architecture, implementation, and deployment of the proposed Intelligent Clinical Document Processing System (ICDPS). The following are the design constraints of ICDPS:

- **Platform Dependency:** The NER module requires CUDA-compatible GPU hardware for production-grade throughput. CPU-only deployment is limited to low-volume testing scenarios.

- **Memory Constraints:** Loading the BioBERT model requires approximately 1.5 GB of GPU VRAM. Simultaneous processing of multiple documents requires proportional VRAM scaling.
- **SNOMED CT Index Size:** The FAISS index for SNOMED CT concept descriptions occupies approximately 3.5 GB of disk space and 6 GB of RAM when loaded.
- **Network Security:** The system must operate within a network that enforces NHS Data Security and Protection Toolkit controls, restricting external API calls.
- **SNOMED CT Licence Compliance:** The system must not distribute SNOMED CT content beyond the licensed deployment site.

Summary

This chapter outlined the software requirements of the ICDPS, covering functional requirements, non-functional requirements, performance targets, operational constraints, and the supporting technologies required for deployment. These requirements define the expected behaviour and operating environment of the proposed system.

The next chapter focuses on the high-level system architecture and explains how the identified requirements influenced the design of the overall processing framework.

Chapter 4

High-Level Design

CHAPTER 4

HIGH-LEVEL DESIGN

The Intelligent Clinical Document Processing System (ICDPS) is organized as a modular architecture that supports document ingestion, information extraction, summarization, and SNOMED CT terminology mapping. Rather than relying on a single monolithic component, the system is divided into a series of interconnected modules, each responsible for a specific stage of the processing workflow. This approach simplifies development, improves maintainability, and allows individual components to be updated independently.

The architecture defines how clinical documents progress through the processing pipeline, beginning with document submission and ending with the generation of structured outputs for review and coding. It also establishes the interactions between the major system components and the technologies that support them.

4.1 Design Considerations

The design of the ICDPS was influenced by both technical and operational requirements associated with clinical document processing. Healthcare documents vary considerably in structure, content, and quality, requiring an architecture capable of handling heterogeneous inputs while maintaining reliable performance. In addition to processing accuracy, considerations such as modularity, scalability, security, maintainability, and future extensibility were incorporated into the design process.

These considerations shaped the overall system architecture, technology selection, deployment approach, and communication mechanisms adopted between the major processing modules.

4.1.1 General Considerations

Several architectural principles influenced the design of the ICDPS. These principles were adopted to ensure that the system could process heterogeneous clinical documents while remaining maintainable, scalable, secure, and reliable throughout its operational lifecycle.

- **Modularity:** The system is organized into independent modules, including Document Ingestion, Text Extraction, NER, Summarization, SNOMED CT Mapping, and Review components. Each module exposes clearly defined inputs and out-

puts, allowing components to be tested, updated, or replaced without affecting the remainder of the pipeline.

- **Scalability:** The processing workflow supports horizontal scaling through containerized deployment. Additional instances can be introduced to accommodate increased document processing workloads when required.

- **Reliability:** Error-handling mechanisms are implemented throughout the pipeline. Documents that cannot be processed successfully are redirected to an error queue for investigation, while processing events are recorded to support auditing and troubleshooting activities.

- **Maintainability:** Source code, configuration files, and model artefacts are managed independently to simplify updates and maintenance. API documentation and version control practices support long-term management of the system.

- **Security:** Access to system interfaces and API endpoints is controlled through role-based authorization mechanisms. Clinical documents and extracted information remain within the deployment environment and are accessible only to authorized users.

- **Explainability:** Generated SNOMED CT suggestions are accompanied by confidence scores, while extracted entities are linked to their corresponding source text spans. These features allow users to review the evidence supporting system outputs before accepting coding recommendations.

4.1.2 Development Methodology

The ICDPS was developed using an iterative Software Development Life Cycle (SDLC) comprising requirements analysis, system design, implementation, testing, and deployment preparation. Rather than completing these activities in a strictly sequential manner, development progressed through multiple iterations, allowing design decisions, implementation choices, and model configurations to be refined as testing results became available.

Project management activities incorporated selected Agile practices, including periodic sprint reviews, version control through GitHub, and continuous integration using

GitHub Actions. These practices supported progress tracking, code quality management, and the coordination of development activities throughout the project.

4.2 Architecture Strategies

Architecture strategies outline the key design approaches adopted for the ICDPS and the manner in which major system components are organised and connected. The architecture combines document processing, NLP, semantic retrieval, and terminology mapping within a modular pipeline that supports maintainability, extensibility, and independent component development.

4.2.1 Technology and Framework Selection

Python 3.10 was selected as the primary implementation language because it provides extensive support for machine learning, natural language processing, and document processing frameworks used throughout the ICDPS pipeline. It also provides compatibility with the frameworks and tools used throughout the project. The Hugging Face Transformers library was selected as the primary NLP framework because it offers well-documented implementations of BioBERT and other transformer-based models, along with standardized interfaces for training and inference.

PyMuPDF was selected for PDF text extraction due to its superior text extraction fidelity on complex clinical PDF layouts compared to alternatives such as pdfplumber and PDFMiner. Tesseract 5.0 was selected for OCR because it supports LSTM-based recognition with NHS-relevant font coverage and is available under a free and open-source licence.

FAISS was selected for SNOMED CT vector search because it provides both exact and approximate nearest-neighbour search with GPU acceleration support, enabling sub-second retrieval from the 1.5 million concept description index. Flask was selected for the web interface due to its lightweight architecture suitable for the single-institution deployment context.

4.2.2 Scalability and Future Enhancements

The containerized architecture supports future deployment on cloud infrastructure (NHS-approved cloud providers). Additional NLP models supporting ICD-10 or LOINC coding could be integrated as independent modules without modifying the core pipeline. The SNOMED CT index can be updated to reflect new SNOMED CT releases by rerun-

ning the index construction script with the new release files.

Future enhancements identified include:

(i) real-time API integration with NHS EHR systems;

(ii) support for handwritten clinical note processing using dedicated handwriting recognition models;

(iii) multilingual support for Welsh and other NHS-relevant languages; and

(iv) active learning integration to continuously improve extraction performance using clinician feedback.

4.3 Overall System Architecture

The Intelligent Clinical Document Processing System (ICDPS) processes clinical documents through a sequence of interconnected stages that transform raw documents into structured clinical information and SNOMED CT coding suggestions. The system is designed to handle documents originating from different sources, including scanned reports, referral letters, discharge summaries, and other healthcare records that may vary in format and quality.

To support this workflow, the architecture is organized into multiple processing layers. Each layer performs a specific task and passes its output to the next stage of the pipeline. This modular structure simplifies development and testing while allowing individual components to be updated or replaced without affecting the overall workflow.

4.3.1 High-Level Architecture Diagram

Figure **??** illustrates the overall processing workflow of the ICDPS. The workflow begins with document ingestion and concludes with the presentation of extracted information, summaries, and coding suggestions through the NHS Portal interface. The architecture is organized into six primary processing layers, each responsible for a distinct stage of document analysis.

Layer 1 — Document Ingestion and Rendering receives uploaded clinical documents through the NHS Portal interface. Documents are processed by the `document_handler` module and routed to PyMuPDF for page-level rendering and conversion into PNG images for downstream processing. **Layer 2 — Image Preprocessing** uses OpenCV to

improve image quality before OCR processing. Operations such as adaptive thresholding, noise reduction, and deskewing are applied to improve text visibility and document consistency.

Layer 3 — OCR Extraction and Confidence Routing uses AWS Textract to extract document text and confidence values. Documents with confidence scores above the predefined threshold continue through the standard processing path, while lower-confidence outputs are forwarded to a LayoutLMv3 refinement stage.

Layer 4 — Entity Extraction and Clinical Coding identifies clinically relevant entities and maps extracted concepts to SNOMED CT terminology using the clinical coding pipeline.

Layer 5 — PHI Detection, Structured Field Extraction, and Summarisation detects protected health information (PHI), extracts structured fields, and generates summaries tailored to different user roles.

Layer 6 — Confidence Aggregation and Output Rendering combines confidence measures from multiple processing stages to calculate a Unified Confidence Score (UCS) and presents the final results through the NHS Portal interface.

4.3.2 Module Description

The ICDPS consists of a set of modules that collectively implement the document processing workflow. Each module is responsible for a specific stage of processing and exchanges information with other components through predefined interfaces. This separation allows individual modules to be modified, optimized, or extended independently as the system evolves.

Table **??** summarizes the major modules in the architecture, along with their inputs, processing responsibilities, and generated outputs.
cccc

Table 4.1: ICDPS Module Descriptions

Module	Input	Processing	Output
`document_handler`	PDF / JPEG / PNG / TIFF	PyMuPDF rendering; file type routing	PNG pa

Module	Input	Processing	Output
OpenCV Preprocessor	Raw page PNGs	Adaptive threshold; MORPH_CLOSE; deskew	Clea
AWS Textract OCR	Preprocessed PNGs	AnalyzeDocument API; block parsing	doc
Confidence Router	`textract_conf`	Threshold comparison vs. 0.90	Rou
LayoutLMv3 Refiner	Low-conf page images	Multi-modal transformer inference	
SNOMED Mapper (4-layer)	`doc_text`; entities; summary	InferSNOMEDCT; regex; lookup table	SN
PHI Detector	`doc_text`	Comprehend Medical DetectPHI	ph
Regex Extractor	`doc_text`	Pattern matching for ICD-10, meds	icd_c
Bedrock Claude LLM	`doc_text` + SNOMED entities	Role-specific prompts; prose generation	
UCS Calculator	`textract_conf`; `snomed_conf`; `llm_conf`	Weighted formula by document type	unifi
NHS Portal UI	JSON result dict	JavaScript / React rendering	Inter

4.3.3 High-Level AI Workflow

The AI workflow within the ICDPS processes extracted document content through multiple analysis paths rather than a single sequential pipeline. After OCR extraction,

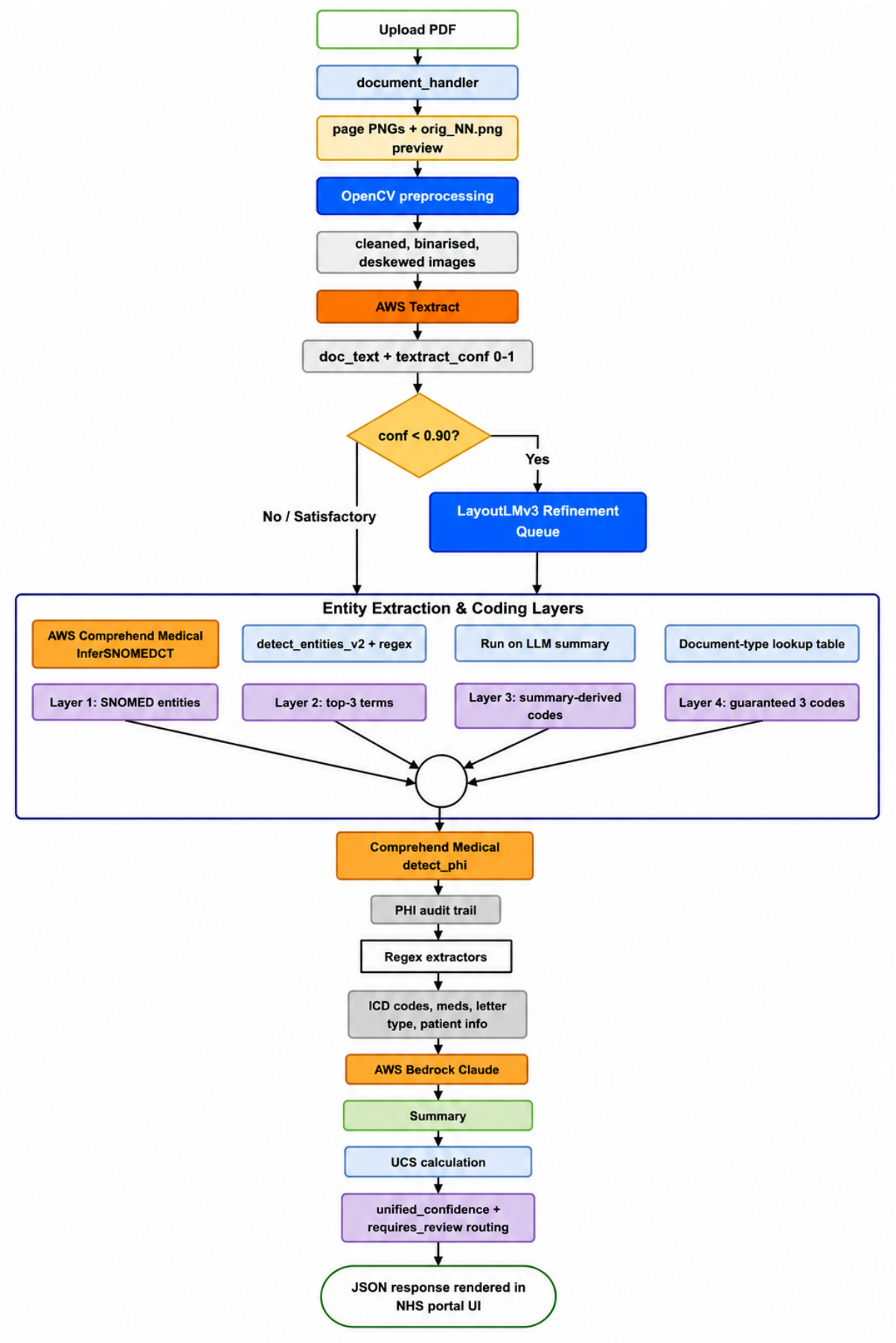

Figure 4.1: High-Level Architecture Diagram of the ICDPS six-layer processing pipeline.

the document content is routed to two independent processing tracks together with a confidence evaluation mechanism.

- **Track A** performs clinical entity extraction and terminology mapping. Extracted medical concepts are processed through the SNOMED CT safety-net architecture to identify appropriate standardized healthcare codes.
- **Track B** performs clinical language understanding and document summarization. AWS Bedrock Claude Sonnet is used to generate contextual summaries tailored to different healthcare users and review scenarios.
- **Confidence Scoring Layer** combines confidence values generated during OCR extraction, SNOMED CT mapping, and language model processing. These values are aggregated into a Unified Confidence Score (UCS), which is used to determine whether the generated output can proceed automatically or requires additional human review.

The separation of entity extraction and language understanding into independent processing tracks allows the system to continue producing useful outputs even when one component performs less effectively on a particular document.

4.4 Data and Learning Workflow

The ICDPS workflow describes how information moves through the system during both training and inference. Clinical documents are progressively transformed from raw uploaded files into structured entities, summaries, and SNOMED CT coding suggestions through a sequence of processing stages.

The workflow includes document ingestion, OCR processing, information extraction, terminology mapping, confidence evaluation, and output generation. Each stage produces intermediate results that are passed to downstream components, allowing individual modules to be evaluated, updated, and maintained independently.

The following sections describe the flow of data through the system and the learning processes used to train and evaluate the machine learning components.

4.4.1 End-to-End Workflow Diagram

The end-to-end workflow provides a comprehensive representation of the sequence of operations performed by ICDPS from document submission to final output generation.

The complete process is divided into five major stages:

1. Document Upload and Ingestion
2. Image Preprocessing
3. OCR Extraction and Confidence Routing
4. Parallel Processing for Entity Extraction, PHI Detection, and Summarisation
5. Confidence Aggregation and Output Rendering

Each stage produces outputs that are passed to downstream components, forming the overall document processing workflow. The process begins when a clinical document is uploaded through the NHS Portal interface. Document pages are rendered and enhanced before OCR extraction converts visual content into machine-readable text. Confidence-based routing is then used to determine whether additional processing through the LayoutLMv3 refinement module is required. After text extraction, multiple downstream tasks execute in parallel to reduce processing time and improve overall throughput.

4.4.2 Training Workflow Overview

Most components within the ICDPS rely on pre-trained models and managed services for OCR, language understanding, and terminology processing. As a result, large-scale model training is not required for every stage of the pipeline. However, the LayoutLMv3 refinement module was adapted for clinical document processing through task-specific fine-tuning.

The training workflow begins with the collection and preparation of NHS-style clinical documents. Documents are converted into annotated page images and associated with bounding-box information representing relevant text regions. The dataset is divided into training, validation, and testing subsets using a 70:15:15 split.

A pre-trained LayoutLMv3-base model is then fine-tuned using the AdamW optimizer together with cosine annealing learning-rate scheduling. Validation performance is evaluated throughout training, and early stopping is used to reduce overfitting. The checkpoint achieving the best validation performance is selected for deployment within the refinement stage of the production pipeline.

4.4.3 Inference Workflow Overview

The inference workflow describes how the deployed system processes newly submitted clinical documents. Unlike the training workflow, no model parameters are updated during this stage.

When a document is uploaded, it undergoes rendering and image preprocessing before being submitted to AWS Textract for OCR extraction. Documents with OCR confidence scores above the predefined threshold of 0.90 continue through the standard processing path. Documents that do not satisfy this threshold are routed to the LayoutLMv3 refinement module for additional processing.

Following text extraction, the SNOMED CT mapping cascade identifies relevant medical concepts and generates standardized terminology suggestions. At the same time, PHI detection, structured field extraction, and document summarization are performed as parallel processing tasks. Outputs from these components are combined with confidence measures produced at different stages of the pipeline to calculate the Unified Confidence Score (UCS).

The UCS provides an overall indication of processing reliability and is used to support downstream workflow decisions. Documents with sufficiently high confidence can proceed directly to output generation, whereas lower-confidence results are flagged for clinician review before final acceptance.

4.5 System Interaction and Deployment Overview

The ICDPS includes both user-facing and backend components that work together to support clinical document processing. Users interact with the platform through document submission and review interfaces, while the underlying infrastructure manages processing tasks, model inference, data storage, and workflow coordination.

This section outlines the user interaction workflow and provides an overview of the deployment environment used to support system operation.

4.5.1 User Interaction Flow

The NHS Portal UI provides the primary point of interaction between healthcare users and the ICDPS. Through this interface, users can upload clinical documents, review generated outputs, and export processed information for further use.

Document submission initiates the backend processing workflow through a multipart

POST request. As the document progresses through the pipeline, processing status is displayed through a progress indicator. Upon completion, results are organized into four tabbed sections: Details, Coding, Follow-up, and GP Actions, allowing users to navigate different categories of extracted information efficiently.

A colour-coded Unified Confidence Score (UCS) badge is displayed alongside the results to indicate overall processing confidence. Users can examine extracted entities, review generated summaries, adjust SNOMED CT coding suggestions when required, and export the resulting structured records for downstream clinical workflows.

4.5.2 Model-Service Interaction

Communication between ICDPS components and supporting cloud services is performed through secure service interfaces. AWS resources are accessed using `boto3` clients over encrypted TLS connections, with related services deployed within the same AWS region to minimise latency.

Protected Health Information (PHI) is excluded from logging operations before audit data is stored. The LayoutLMv3 refinement component is deployed as a containerized local service, enabling independent inference and asynchronous processing of documents that require additional analysis.

4.5.3 Deployment Overview

The ICDPS adopts a cloud-native deployment architecture that combines containerized application services with managed AWS infrastructure. The Flask backend and LayoutLMv3 inference component are packaged as Docker containers and deployed using Amazon ECS with Fargate support.

The NHS Portal frontend is distributed through CloudFront to improve availability and reduce response times. Amazon SQS facilitates asynchronous communication between processing stages, while AWS Step Functions orchestrate workflow execution and error-handling operations. Audit records, extracted information, and processing metadata are stored in Amazon DynamoDB.

By distributing responsibilities across multiple services, the deployment architecture supports scalability, fault tolerance, and flexible management of document processing workloads.

Summary

This chapter presented the high-level design of the Intelligent Clinical Document Processing System (ICDPS), outlining the major processing layers, module interactions, AI workflow, deployment architecture, and user interaction model. The architectural principles and technology choices that influenced the overall system design were also discussed.

The high-level design establishes how information flows through the system, from document ingestion and processing to structured output generation and review. The next chapter focuses on the detailed design of individual components, including data organisation, preprocessing procedures, model configurations, and implementation-specific design considerations.

Chapter 5

Detailed Design

CHAPTER 5

DETAILED DESIGN

This chapter presents the detailed design of the Intelligent Clinical Document Processing System (ICDPS). Building upon the high-level architecture described in the previous chapter, it examines the internal structure of individual components, the data flow between processing stages, and the design decisions that support system implementation.

The chapter covers dataset organisation, document preprocessing, model architecture, terminology mapping, training procedures, and inference workflows. Collectively, these elements define the technical processes through which clinical documents are transformed into structured information and SNOMED CT coding recommendations.

5.1 Dataset and Data Organisation

The implementation of the ICDPS required data resources supporting document understanding, clinical information extraction, and terminology mapping tasks. Effective organisation of these resources was necessary to support preprocessing, model development, evaluation, and validation activities throughout the project.

The datasets used in this work comprise clinical documents, annotation files, and supporting terminology resources. These datasets were organised to facilitate model training, testing, and performance assessment while maintaining a clear separation between development and evaluation data. This section describes the composition of the datasets, their organisational structure, and the data preparation procedures adopted during system development.

5.1.1 Dataset Description

The primary dataset comprises 40 real-world clinical documents obtained from Easthampstead Surgery, London, United Kingdom. These documents represent the full spectrum encountered in a typical NHS primary care workflow: outpatient correspondence, specialist referral letters, hospital discharge summaries, NHS 111 reports, Accident and Emergency summaries, ambulance service reports, diabetic eye screening reports, and out-of-hours primary care reports. The dataset includes both digitally generated PDF documents and scanned document images produced from photographed or photocopied clinical letters.

Each document contains structured data fields (patient NHS number, date of birth,

consultant name, department) and unstructured free-text clinical narrative sections describing presenting complaints, diagnoses, management plans, medication prescriptions, and follow-up recommendations. The dataset is annotated with ground-truth SNOMED CT codes for 25 documents, enabling quantitative evaluation of SNOMED mapping accuracy. Table **??** summarises the key dataset characteristics.

Table 5.1: Dataset Characteristics Summary

Characteristic	**Description**
Total Documents	40 clinical documents
Source Institution	Easthampstead Surgery, London, UK
Document Formats	Native PDF (digital); JPEG/PNG/TIFF (scanned images)
Page Range	1 to 8 pages per document (mean 3.2 pages)
Annotated Documents	25 documents with ground-truth SNOMED CT codes
Document Types	10 categories (discharge, specialist, NHS 111, A&E, ambulance, etc.)
Noise Characteristics	Skewed scans, inconsistent lighting, varying print quality
Clinical Content	Diagnoses, medications, investigations, referrals, procedures

5.1.2 Data Collection and Sources

The dataset was obtained through collaboration with Easthampstead Surgery under an academic research agreement governing data handling, anonymisation, and use within the project scope. All documents were de-identified prior to use, with patient names, NHS numbers, dates of birth, addresses, and other PHI replaced with synthetic identifiers, consistent with NHS Information Governance requirements.

Document collection involved scanning physical paper records at 300 dpi using an NHS-compliant document scanner and exporting digital records from the document management system in PDF format. Additional scanned copies were generated by photographing physical documents under controlled lighting conditions to simulate mobile image capture scenarios commonly encountered in clinical environments. These images incorporated variations in skew, noise levels, contrast, and illumination conditions to reflect the diversity of document quality encountered in real-world clinical environments.

Including documents with diverse image characteristics enabled the preprocessing and OCR components to be assessed under a range of realistic operating conditions rather than being evaluated solely on clean, digitally generated documents.

5.1.3 Dataset Structure and Characteristics

The dataset encompasses documents exhibiting substantial variation in layout, formatting conventions, font styles, and structural organisation. While some records follow predefined templates with clearly identifiable sections and structured data fields, others are composed largely of free-text clinical narratives with limited visual organisation.

Such diversity is characteristic of clinical documentation originating from different healthcare environments and introduces additional complexity for automated information extraction. The presence of heterogeneous document structures influenced the design of the extraction framework, leading to the adoption of a multi-stage processing pipeline rather than a template-dependent approach that would be less adaptable across the range of document formats represented in the dataset.

Content analysis of the 40 documents identified the following clinical entity categories: diagnostic conditions (62% of documents), medications with dosages (78%), physiological measurements (45%), clinical procedures (38%), anatomical locations (71%), and temporal references (83%). The class distribution of SNOMED CT codes in the annotated subset is dominated by disorder/finding concepts (58%) and drug substance concepts (29%), with procedure (9%) and body structure (4%) concepts forming the minority. Table **??** presents the distribution of document types.

Table 5.2: Distribution of Document Types in the ICDPS Dataset

Document Type	**Count**	**Percentage (%)**
Hospital Discharge Summaries	8	20.0
Specialist Clinical Letters	9	22.5
NHS 111 Reports	5	12.5
Accident and Emergency Reports	4	10.0
Ambulance Service Reports	4	10.0
Private Specialist Letters	3	7.5
Diabetic Eye Screening Reports	3	7.5
Out-of-Hours Primary Care Reports	2	5.0
External Provider Reports (Eye/ENT)	2	5.0
Total	**40**	**100.0**

5.2 Preprocessing Pipeline Design

Data preprocessing plays a critical role in improving document quality prior to Optical Character Recognition (OCR) and directly influences the accuracy of downstream

clinical entity extraction and coding. Clinical documents frequently contain variations in scan quality, skewed pages, uneven lighting, handwritten notes, stamps, and background artefacts that reduce OCR performance. Therefore, a structured preprocessing pipeline was implemented to improve document readability and ensure consistent input quality before processing with Amazon Textract.

5.2.1 Data Cleaning

Collected clinical documents were reviewed and preprocessed before OCR execution. Blank or nearly empty pages were removed to avoid unnecessary processing overhead. Documents were converted to a standardized RGB image format to ensure compatibility with OpenCV image operations and Amazon Textract input requirements. Image dimensions and resolution were also normalized to reduce variations introduced by different scanning devices and document sources.

Pages containing severe distortions, incomplete scans, or excessive artefacts were identified during preprocessing. Documents with image quality issues likely to cause substantial OCR errors were excluded from quantitative evaluation to prevent unreliable results from affecting performance measurements.

5.2.2 Noise Reduction and Handling Missing Information

Clinical documents often contain handwritten annotations, photocopy artefacts, ink stamps, variations in paper quality, and scanning-related noise that can affect document readability and downstream text extraction processes. These factors can affect OCR accuracy and reduce the quality of downstream information extraction. To reduce their impact, image enhancement techniques implemented with OpenCV were applied before text extraction.

Morphological filtering operations were used to suppress isolated noise while preserving character structures and connected text components. Adaptive Gaussian thresholding was applied to improve text visibility under varying lighting and scanning conditions. These operations increased text contrast and improved OCR readability.

When specific information could not be extracted reliably, missing values were retained as null fields rather than estimated. Similarly, the summarization component was configured to indicate unavailable information instead of generating unsupported clinical content.

5.2.3 Text Transformation and Normalization

Following noise reduction, image transformations were applied to improve OCR consistency. Documents were converted to grayscale representations and processed using adaptive binarization to increase foreground text visibility. Page skew correction was performed by estimating the dominant text orientation and rotating pages to align text horizontally. Rotation limits were applied to prevent excessive corrections on pages containing limited textual content.

After OCR extraction, text-level normalization operations were performed before NLP processing. Multi-page document outputs were merged into a unified text representation while preserving page boundaries. Repeated headers and footers generated during OCR extraction were removed to reduce redundancy. Unicode characters were normalized into a consistent format to ensure compatibility with downstream components. Basic sentence-boundary corrections were also applied where OCR-induced line breaks interrupted sentence continuity.

These preprocessing operations reduce OCR-related noise and provide more consistent input for entity extraction, terminology mapping, and summarization components.

5.2.4 Tokenization and Label Alignment

Clinical text is tokenized using the BioBERT WordPiece tokenizer with a maximum sequence length of 512 tokens. During model fine-tuning, entity annotations from the curated clinical document dataset are aligned with the generated subword token sequence using a token-label mapping strategy. The first subword token corresponding to an annotated entity receives the appropriate B- or I- label according to the BIO tagging scheme, while subsequent subword tokens belonging to the same word receive an X label indicating continuation.

Tokens assigned the X label are excluded from loss computation during training. This approach allows the model to learn entity boundaries from the original word-level annotations without introducing duplicate supervision signals for subword fragments.

For documents exceeding 512 tokens, a sliding-window strategy is applied using a window size of 512 tokens and a stride of 256 tokens. Duplicate predictions within overlapping regions are resolved by retaining predictions from the window in which the token appears within the non-overlapping section. If the confidence score of an overlapping prediction exceeds the retained prediction by more than 15

5.2.5 Data Augmentation

To increase the representation of Allergy and Dosage entities in the training data, two augmentation techniques were applied:

(i) **Entity Synonym Replacement:** Clinical entities were replaced with equivalent terms obtained from SNOMED CT synonym lists, generating additional training examples while preserving the original annotations.

(ii) **Back-Translation:** Sentences containing underrepresented entities were translated into French and then translated back into English using a neural machine translation model. This process generated paraphrased variants while preserving the original clinical meaning.

5.3 NER Model Architecture Design

The Named Entity Recognition (NER) component is responsible for identifying clinically relevant information from unstructured medical text. Clinical documents often contain abbreviations, specialized terminology, incomplete sentences, and context-dependent expressions that make entity extraction challenging. To address these characteristics, the proposed system uses a BioBERT-based architecture that generates contextual representations of clinical text and predicts entity labels at the token level.

The model combines transformer-based contextual embeddings with sequence-level classification mechanisms to identify diagnoses, medications, procedures, allergies, laboratory findings, and other clinical entities. Additional contextual analysis is applied during post-processing to distinguish affirmed, negated, and uncertain clinical statements, improving the quality of extracted information before SNOMED CT mapping and downstream processing.

5.3.1 BioBERT Encoder

The NER encoder is BioBERT-base (cased), comprising 12 transformer encoder layers, 12 attention heads per layer, a hidden dimension of 768, and a total of 110 million parameters. The model processes tokenized input and produces a sequence of contextual embeddings of dimension 768 for each token. The `[CLS]` token embedding is not used for NER; only the per-token embeddings are passed to the classification head.

5.3.2 CRF Classification Head

A linear classification layer maps each 768-dimensional token embedding to a vector of logits over the label vocabulary (17 labels: B- and I- for each of 8 entity types, plus O). A Conditional Random Field (CRF) layer is applied on top of the linear layer to enforce label sequence constraints. The CRF layer learns a transition score matrix T where $T[i][j]$ represents the learned score of transitioning from label i to label j. During inference, the Viterbi algorithm decodes the optimal label sequence.

The CRF layer enforces the following hard constraints:

- I-TYPE labels cannot follow O labels without an intervening B-TYPE label.
- I-TYPE labels of one entity type cannot follow B-TYPE or I-TYPE labels of a different entity type.
- B-TYPE and I-TYPE labels of the same entity type can follow each other without restriction.

5.3.3 Negation Detection Module

The negation detection module determines whether an extracted clinical entity is affirmed, negated, or expressed with uncertainty. For each identified entity, a context window of ± 5 tokens surrounding the entity span is extracted from the tokenized document. BioBERT embeddings generated for the context window are passed to a linear classification layer that predicts one of three assertion states: *Affirmed*, *Negated*, or *Uncertain*.

The classifier is initialized using NegEx trigger phrase lists and subsequently adapted using assertion annotations derived from the curated clinical document dataset and NHS-representative records. This combination of rule-based initialization and supervised learning allows the model to recognize both explicit negation triggers and more complex contextual expressions of uncertainty.

5.4 SNOMED CT Mapping Module Design

The SNOMED CT mapping module is responsible for linking extracted clinical entities to standardized terminology concepts. This task is complicated by the variability of clinical language, where the same concept may be represented using synonyms, abbreviations, alternative spellings, or context-specific expressions. Consequently, exact keyword

matching alone is often inadequate for reliable terminology assignment. To address this challenge, the mapping framework employs a combination of semantic retrieval and candidate ranking techniques to identify the SNOMED CT concepts most closely aligned with the meaning of the extracted entity.

The proposed framework represents clinical entities and SNOMED CT descriptions using vector embeddings. Candidate concepts are retrieved through similarity-based search and subsequently ranked using contextual information derived from the source document. This approach enables the system to generate confidence-ranked coding suggestions while supporting large-scale terminology collections containing hundreds of thousands of concepts.

5.4.1 Concept Embedding Index Construction

The SNOMED CT concept description index is constructed as follows:

(i) All active concepts in the UK SNOMED CT release are extracted.

(ii) For each concept, its preferred term, fully specified name, and all active synonyms are concatenated into a description string.

(iii) FastText embeddings are computed for each description string using a character n-gram model ($n = 3$–6) pre-trained on a biomedical text corpus.

(iv) The resulting embedding vectors (dimension 300) are indexed in a FAISS `IndexFlatIP` (inner product) index for exact search, and a FAISS `IndexIVFPQ` index for approximate search.

5.4.2 Candidate Retrieval and Reranking

For each affirmed entity, the following two-stage retrieval process is performed:

- **Stage 1 — Candidate Retrieval:** The entity text is encoded using the FastText model to produce a query vector. The top-50 nearest neighbours (by inner product similarity) are retrieved from the FAISS index, producing 50 candidate SNOMED CT concepts.

- **Stage 2 — Reranking:** The 50 candidates are reranked using a cross-encoder model — a fine-tuned BioBERT model that takes as input the concatenation of the

entity span (with its surrounding context) and the SNOMED CT concept description, and outputs a relevance score. The top-5 candidates by reranking score are returned as the final suggestions with associated confidence scores.

5.4.3 Confidence Score Calibration

Raw reranker scores are calibrated to produce reliable probability estimates using Platt scaling, fitting a logistic regression model on the reranker scores and binary relevance labels from a held-out validation set. Calibrated confidence scores are verified to be within 5% of empirical accuracy across five confidence deciles on the validation set.

5.5 Model Architecture and Experimental Setup

The ICDPS integrates multiple AI/ML model components, each addressing a distinct processing challenge. This section presents the architecture of each model component and the experimental setup adopted for training and evaluation.

5.5.1 Model Selection

Model selection was guided by three criteria: (i) clinical domain suitability, favouring models pre-trained on biomedical or clinical text; (ii) service integration feasibility, preferring managed AWS services; and (iii) published performance benchmarks on comparable tasks. Table **??** presents the candidate models considered and the selection rationale for each pipeline component.

ccc

Table 5.3: Model Selection Analysis for ICDPS Components

Component	**Candidates Considered**	**Selected Model and Rationale**
OCR Engine	AWS Textract; Tesseract 5; Google Cloud Vision	AWS Textract — superior table/form extraction; per-word confidence scoring; HIPAA-eligible
Layout Understanding	LayoutLMv3; LayoutXLM; DocFormer	LayoutLMv3 — state-of-the-art on PubLayNet/FUNSD; multi-modal text + layout + image
Medical NER	AWS Comprehend Medical; BioBERT; spaCy scispaCy	AWS Comprehend Medical — managed HIPAA-eligible; native InferSNOMEDCT API

Component	Candidates Considered	Selected Model and Rationale
Clinical LLM	AWS Bedrock Claude Sonnet; GPT-4; Llama 3	AWS Bedrock Claude Sonnet — best instruction following; lowest hallucination rate; HIPAA-eligible
Concept Embedding	SapBERT; BioBERT-NER; ClinicalBERT	SapBERT — purpose-designed for medical entity normalisation on SNOMED concepts
Vector Search	FAISS; Pinecone; Milvus	FAISS — open-source; GPU-accelerated; sub-100 ms retrieval

5.5.2 Model Architecture Design

LayoutLMv3 Architecture: LayoutLMv3 is a unified multi-modal pre-trained model for document understanding. The model architecture consists of a shared transformer encoder with 12 layers, 12 attention heads, and a hidden dimension of 768 (LayoutLMv3-base). Text tokens are represented as WordPiece embeddings summed with 1D position embeddings and 2D spatial layout embeddings derived from word bounding box coordinates. Image patches are encoded using a linear projection of 16×16 pixel

patches from the document page image. Cross-modal attention between text token representations and image patch representations enables the model to associate text with its visual context on the page. In the ICDPS, LayoutLMv3 is fine-tuned for token classification into five layout-semantic categories: Title, Section Header, Body Text, Data Field, and Table Cell.

AWS Comprehend Medical NER: The `InferSNOMEDCT()` API operates on a clinical BERT-based sequence labeller pre-trained on MIMIC-III clinical notes, PubMed abstracts, and UMLS metathesaurus concept descriptions. The model applies a CRF output layer to produce entity span labels in BIO format. Each detected entity is associated with a SNOMED CT concept ID, a description string, a clinical category, and a detection confidence score.

AWS Bedrock Claude Sonnet: Claude Sonnet provides a 200,000-token context window, enabling processing of multi-page clinical documents without truncation. Three role-differentiated system prompts are constructed at runtime: a Clinician System Prompt for structured clinical summaries covering findings, diagnoses, and recommended actions; a Patient System Prompt for plain-English explanation avoiding medical jargon; and a Pharmacist System Prompt focusing on medication details, dosages, and contraindications.

5.5.3 Training Configuration

The LayoutLMv3 fine-tuning training configuration was established through a preliminary hyperparameter grid search. Table **??** presents the final training configuration adopted.

5.5.4 Experimental Workflow

The experimental workflow followed structured training, evaluation, and hyperparameter refinement cycles. In each cycle: (i) LayoutLMv3 was fine-tuned on the augmented training dataset; (ii) the fine-tuned model was evaluated on the validation set using token-level accuracy, precision, recall, and F1-score for each zone class; (iii) training and validation loss curves were inspected for overfitting; (iv) hyperparameters were adjusted based on validation performance; and (v) the best checkpoint by macro F1-score was retained. For SNOMED mapping, layer-by-layer ablation experiments quantified the marginal contribution of each fallback layer to overall SNOMED code recall. UCS

weighting coefficients were validated through threshold sensitivity analysis across the full confidence threshold range $[0.60, 0.80]$ for all 21 document type categories.

Table 5.4: LayoutLMv3 Fine-Tuning Training Configuration

Hyperparameter	**Value**
Base Model	`microsoft/layoutlmv3-base` (Hugging Face Hub)
Task	Token Classification — document layout segmentation
Training Epochs	15 (early stopping patience: 5 epochs)
Batch Size	8 (gradient accumulated over 4 steps; effective batch = 32)
Optimiser	AdamW
Initial Learning Rate	2×10^{-5}
Learning Rate Schedule	Cosine annealing with warm-up (warmup ratio = 0.1)
Weight Decay	0.01
Dropout Rate	0.1
Max Input Sequence Length	512 tokens
Image Resolution	224×224 pixels (ViT patch size 16×16)
Hardware	NVIDIA A10G GPU (AWS g5.xlarge)
Training Duration	Approximately 4.5 hours for 15 epochs
Framework	PyTorch 2.1 + Hugging Face Transformers 4.37

5.5.5 Validation Strategy

The primary validation strategy for SNOMED CT mapping accuracy employs leave-one-out cross-validation (LOOCV) across the 25 annotated documents, reporting precision, recall, and F1-score at the entity-code pair level. LOOCV was selected because the small dataset size makes k-fold partitions unstable for minority document categories. For LayoutLMv3 zone classification, stratified 5-fold cross-validation on the synthetic training dataset reports token-level macro F1-score. OCR accuracy is evaluated using character error rate (CER) and word error rate (WER) computed against manually verified ground-truth transcriptions of 15 representative document pages.

5.6 Inference and Post-Processing Design

After model training, the ICDPS processes clinical documents through an inference pipeline that generates entity predictions, terminology mappings, summaries, and structured outputs. In addition to model inference, several post-processing stages are applied to improve output quality and ensure consistency across downstream components.

These stages encompass prediction refinement, confidence assessment, terminology ranking, and the generation of structured outputs suitable for downstream processing and review. Together, they transform raw model predictions into outputs that can be reviewed and used within clinical workflows.

5.6.1 Optimisation Techniques

LayoutLMv3 fine-tuning uses the AdamW optimizer, which separates weight decay from the adaptive gradient update process and is widely used for transformer-based models. A weight decay value of 0.01 is applied to all parameters except bias terms and layer-normalization parameters. Learning-rate warm-up is performed over the first 10

Cosine annealing is used to gradually reduce the learning rate throughout training, allowing the model to converge more smoothly. Gradient clipping with a maximum norm of 1.0 limits excessively large gradient updates. Mixed-precision training (FP16) reduces GPU memory consumption by approximately 40

5.6.2 Loss Functions and Evaluation Logic

The LayoutLMv3 token-classification model is trained using a weighted cross-entropy loss function. Class weights are assigned inversely proportional to class frequency in order to reduce the impact of class imbalance between frequently occurring Body Text tokens and less common classes such as Table Cell and Section Header tokens.

The weighted loss function, denoted by $\mathcal{L}_w$, is defined as:

$$\mathcal{L}_w = -\sum_c [w_c \cdot y_c \cdot \log(p_c)] \quad \text{where} \quad w_c = \frac{N_{\text{total}}}{N_{\text{classes}} \cdot N_c} \tag{5.1}$$

The Unified Confidence Score (UCS) is computed using document-type-dependent weighted linear combinations. For standard NHS documents (discharge summaries, specialist letters):

$$\text{UCS} = 0.40 \times \text{textract\_conf} + 0.20 \times \text{snomed\_conf} + 0.40 \times \text{llm\_conf} \tag{5.2}$$

For ambulance and NHS 111 reports, where all-caps tables render SNOMED mapping unreliable:

$$\text{UCS} = 0.50 \times \text{textract\_conf} + 0.50 \times \text{llm\_conf} \tag{5.3}$$

The UCS is compared against document-type-specific thresholds defined in `config/document_type_config.py`, which defines 21 NHS document categories with per-type review thresholds ranging from 0.62 to 0.72.

5.6.3 Inference Pipeline

The production inference pipeline is implemented as a Python class (`IcdpsInferencePipeline`) that encapsulates the complete processing workflow. The pipeline accepts a document path as input and executes the following steps in sequence: document format detection, text extraction (direct or OCR), text normalization, sliding-window tokenization, NER inference, entity span decoding, negation detection, role categorization, summarization, SNOMED CT candidate retrieval, and reranking. The pipeline returns a structured JSON object containing the document metadata, extracted entities, summary, and SNOMED CT suggestions.

5.6.4 Post-Processing Rules

The following rules are applied to NER predictions before structured output generation:

- **Contiguous Entity Merging:** Adjacent entities of the same type separated only by whitespace or the conjunction 'and" are merged into a single entity span. For example, the sequence 'left" + 'ventricular" + 'hypertrophy" is merged into the entity "left ventricular hypertrophy".
- **Minimum Entity Length Filter:** Entity spans containing fewer than two non-whitespace characters are removed, as these are typically generated by tokenization or OCR artefacts.
- **Duplicate Entity Deduplication:** Repeated occurrences of the same entity text within a document are consolidated. The first occurrence and its associated SNOMED CT mapping are retained in the structured output, while all occurrences remain highlighted within the document view.
- **Negated Entity Exclusion:** Entities classified as *Negated* by the assertion detection module are excluded from SNOMED CT code generation. These entities remain visible to the user but are clearly marked and omitted from exported coding reports.

5.6.5 Output Schema

The inference process produces a structured JSON output containing extracted entities, SNOMED CT mappings, confidence scores, generated summaries, and associated metadata. The structure of the output is illustrated in the following schema:

```
{
  "document_id": string,
  "document_type": string,
  "extraction_timestamp": ISO8601,
  "entities": [{
    "entity_id": string,
    "text": string,
    "category": string,
    "assertion": string,
    "role_action": string,
    "start_char": int,
    "end_char": int,
    "snomed_suggestions": [{
      "concept_id": string,
      "preferred_term": string,
      "confidence": float
    }]
  }],
  "summary": string,
  "processing_metadata": {
    "model_version": string,
    "processing_time_ms": int
  }
}
```

Summary

This chapter presented the detailed design of the Intelligent Clinical Document Processing System (ICDPS), covering dataset organisation, preprocessing workflows, data

augmentation methods, model architecture, terminology mapping procedures, and inference workflows. It also described the post-processing mechanisms and confidence-based evaluation strategies used to improve the reliability and consistency of generated outputs.

Key aspects of the design included the BioBERT-based clinical entity extraction framework, the SNOMED CT retrieval and ranking process, and the handling of lengthy clinical documents through sliding-window processing. Training configurations, optimisation approaches, and output-generation mechanisms were also detailed to establish the technical foundation for system implementation.

Chapter 6 focuses on the implementation of the proposed design and discusses the technologies, frameworks, and development practices used during system construction.

Chapter 6

Implementation Details

CHAPTER 6

IMPLEMENTATION DETAILS

The implementation of the Intelligent Clinical Document Processing System (ICDPS) involved integrating document processing, machine learning, semantic retrieval, and clinical coding components into a unified workflow. The system combines OCR services, transformer-based language models, terminology mapping mechanisms, and web-based interfaces to process clinical documents and generate structured outputs.

This chapter describes the practical implementation of the proposed system, including the development environment, software architecture, model integration procedures, training workflows, and deployment components. The technologies, frameworks, and tools used during development are presented along with implementation details for the major modules of the processing pipeline.

In addition, the chapter discusses the evaluation procedures adopted during development and highlights implementation challenges encountered while building and integrating the different system components.

6.1 Implementation Workflow

The implementation of the ICDPS followed a phased workflow aligned with the iterative SDLC adopted for the project. Each phase produced verifiable outputs that were validated before proceeding to the next phase. The workflow comprised ten sequential steps from environment setup through post-deployment monitoring.

6.1.1 Environment Setup

The development environment was configured on a workstation running Ubuntu 22.04 LTS equipped with an NVIDIA RTX 3090 GPU (24 GB VRAM). The following installation sequence was followed:

1. Python 3.10 was installed via the deadsnakes PPA and a virtual environment was created using `python3.10 -m venv icdps_env` to isolate project dependencies.

2. CUDA 11.8 and cuDNN 8.6 were installed from the NVIDIA developer portal to enable GPU-accelerated PyTorch operations.

3. PyTorch 2.0 with CUDA 11.8 support was installed.

4. The Hugging Face Transformers library (v4.38), Datasets library (v2.17), and PEFT library (v0.8) were installed for model fine-tuning and evaluation.

5. Tesseract 5.0 OCR engine was installed.

6. PyMuPDF (v1.23), FAISS-GPU (v1.7.4), Flask (v3.0), SQLAlchemy (v2.0), and PostgreSQL 16 client libraries were installed.

7. Docker 24 and Docker Compose v2 were installed for containerized deployment preparation.

8. Jupyter Lab was configured for exploratory data analysis and training experiment notebooks; Visual Studio Code with the Python and Pylance extensions served as the primary IDE for production code development.

All project dependencies were maintained in a `requirements.txt` file with fixed version numbers to ensure consistent package installation across development, testing, and deployment environments. For deployment, the complete runtime environment was packaged as a Docker image containing all required libraries, configurations, and application components.

6.1.2 Data Collection

The dataset used in this work was developed to represent the variety of clinical documents encountered in NHS workflows. Documents differed in layout, formatting, image quality, and terminology usage, providing a realistic test environment for document processing and information extraction tasks.

The primary dataset consisted of 40 NHS clinical documents obtained from Easthampstead Surgery through a formal academic research data-sharing agreement. All data handling procedures followed institutional research and data-governance requirements. The collection included both digitally generated records and scanned paper documents, reflecting the mixed-document environments commonly found in NHS clinical practice.

Paper records were digitized using a Canon imageRUNNER scanner at a resolution of 300 dpi. Digital records stored within the practice document management system were exported directly as PDF files to preserve their original formatting and image quality.

Although the real-world dataset was suitable for evaluation, the limited number of annotated documents restricted its usefulness for training document-understanding mod-

els such as LayoutLMv3. To increase training data availability, a synthetic document generation pipeline was developed using the Python ReportLab library.

Approximately 800 synthetic NHS-style clinical letter images were generated using layouts and document structures derived from the real document collection. To simulate practical scanning conditions, the generated documents were processed using ImageMagick-based degradation techniques that introduced blur, brightness variation, contrast changes, and image noise. These synthetic samples provided additional variability during model training and improved robustness to differences in document quality.

Ground-truth SNOMED CT annotations were created for 25 evaluation documents through manual review by a clinical domain expert using the NHS SNOMED CT Browser. These annotations were used for extraction evaluation and terminology-mapping assessment.

6.1.3 Data Preprocessing

Before OCR processing, all documents passed through an image-preprocessing pipeline designed to improve text visibility and reduce image-quality issues that could affect downstream extraction tasks. Common problems included skewed pages, uneven illumination, photocopy artefacts, and scanning noise.

The preprocessing pipeline was implemented as a standalone Python module (`preprocessing/image`
using OpenCV-based image enhancement operations.

The first stage applied adaptive thresholding to improve text contrast and compensate for local lighting variations. OpenCV's `cv2.adaptiveThreshold()` function was configured with:

- Method: `cv2.ADAPTIVE_THRESH_GAUSSIAN_C`
- Block size: 11
- Constant parameter: 2

Unlike global thresholding methods, adaptive thresholding calculates threshold values from neighbouring pixel intensities. Gaussian-weighted averaging across surrounding 11×11 pixel regions was used to determine local threshold values for each image location.

This approach improved image quality in cases involving:

- Uneven illumination

- Curved document pages
- Shadows introduced during handheld image capture
- Localized brightness variation

After thresholding, morphological filtering was applied to reduce noise while preserving character structure. OpenCV's `cv2.morphologyEx(MORPH_CLOSE)` operation was used with a 2×2 rectangular structuring element. The closing operation performs dilation followed by erosion, helping remove isolated artefacts and reconnect broken character strokes.

The operation improved OCR readiness by:

- Removing background speckles
- Reducing photocopy artefacts
- Connecting fragmented character strokes
- Preserving text continuity

The final preprocessing stage involved document deskewing. Rotational distortions introduced during scanning or image capture can reduce OCR accuracy and affect document layout analysis. Text orientation was estimated using contour-based analysis:

1. Contours were detected using `cv2.findContours()`
2. Bounding boxes were computed using `cv2.minAreaRect()`
3. Orientation angles were extracted from text regions
4. A median angle estimate was used to determine page rotation

After angle estimation, images were rotated using `cv2.warpAffine()` with bicubic interpolation. Rotation corrections were restricted to $\pm 15^{\circ}$ to prevent excessive transformations on pages containing limited text content or sparse document structure.

6.1.4 Feature Engineering

Feature engineering was performed to transform OCR outputs into structured and semantically meaningful representations suitable for downstream extraction and coding tasks. Since OCR systems frequently introduce inconsistencies such as broken sentence structures, irregular spacing, and encoding artefacts, additional normalisation procedures were necessary before passing extracted text into subsequent NLP components.

Feature engineering operations were implemented within the `extraction/text_normaliser.py` module and executed sequentially.

The first transformation involved Unicode normalisation using `unicodedata.normalize('NFKD')`. Unicode decomposition reduced inconsistencies arising from special symbols and character encoding differences across document sources.

The second stage expanded NHS-specific abbreviations using a manually curated dictionary containing 847 clinical abbreviations. Clinical documentation frequently contains shorthand terminology such as:

- BP → Blood Pressure
- SOB → Shortness of Breath
- HR → Heart Rate

Expansion reduced ambiguity and improved downstream entity extraction performance. Subsequent transformations included:

- **Page boundary insertion:** Explicit markers were inserted between concatenated multi-page text blocks to preserve structural information and prevent contextual mixing across pages.
- **Whitespace normalisation:** Repeated spaces, tabs, and newline characters were collapsed into single spaces to ensure consistent text formatting.
- **Sentence boundary correction:** OCR systems often incorrectly split sentences across multiple lines. Sentence boundary normalisation inserted punctuation at locations where line breaks interrupted incomplete sentence structures.

For semantic clinical concept matching, extracted entities identified by AWS Comprehend Medical were passed into the SapBERT pipeline. Clinical terms were tokenised and

encoded into 768-dimensional biomedical embeddings representing semantic relationships between concepts. These embeddings were stored and queried using the FAISS nearest-neighbour search framework, enabling efficient retrieval of semantically related clinical concepts and improving SNOMED code matching accuracy.

6.1.5 Model Selection

Models for each stage of the ICDPS pipeline were selected following comparative evaluation using representative NHS clinical documents. The objective was to identify approaches that provided a suitable balance between accuracy, robustness, and deployment practicality.

OCR benchmarking compared AWS Textract and Tesseract 5.0 on both standard and degraded clinical documents. AWS Textract achieved consistently higher extraction accuracy across both document categories:

- Standard document images: AWS Textract 94.7
- Degraded scanned documents: AWS Textract 87.2

The performance difference was particularly noticeable on low-quality scans containing noise, skew, and contrast variations. Based on these results, AWS Textract was selected as the primary OCR engine for the production pipeline.

For document understanding tasks, LayoutLMv3 was evaluated against LayoutLMv2 and a rule-based document classification approach. Evaluation results were:

- LayoutLMv3: Macro F1 = 0.891
- LayoutLMv2: Macro F1 = 0.824
- Rule-based classifier: Macro F1 = 0.712

LayoutLMv3 achieved the highest overall performance and was therefore selected for document understanding and refinement tasks. Its ability to jointly process textual and visual information proved beneficial when handling diverse clinical document layouts.

For clinical entity extraction and terminology mapping, AWS Comprehend Medical InferSNOMEDCT() was compared with a BioBERT-based NER pipeline. The evaluation produced the following results:

- Precision: Comprehend Medical = 0.823; BioBERT = 0.761
- Recall: Comprehend Medical = 0.714; BioBERT = 0.742

Although the BioBERT model achieved slightly higher recall, AWS Comprehend Medical provided higher precision. Since incorrect clinical code suggestions may require additional review and correction, precision was prioritised during model selection. Consequently, AWS Comprehend Medical was adopted as the primary terminology extraction component.

6.1.6 Model Training

The LayoutLMv3 refinement component required adaptation to the document structures and formatting characteristics observed in NHS clinical records. Fine-tuning was therefore performed using a combination of real and synthetic document data.

Training was implemented using the Hugging Face Trainer API through the script `training/finetune_layoutlmv3.py`.

The final training corpus contained approximately 850 document images derived from both real and synthetic sources. Dataset allocation consisted of:

- Training set: 680 documents (80%)
- Validation and testing set: 170 documents (20%)

Training was conducted on an AWS g5.xlarge instance equipped with GPU acceleration. The complete training process required approximately 4.5 hours and was configured to run for a maximum of 15 epochs.

Training metrics showed stable convergence throughout the optimization process:

- Epoch 1 loss: 1.423
- Epoch 12 loss: 0.187
- Minimum validation loss: 0.214 at epoch 10

After epoch 12, validation loss increased while training loss continued to decrease, indicating the onset of overfitting. Early stopping with a patience value of five epochs was applied, and the checkpoint corresponding to the lowest validation loss was retained for deployment. This checkpoint was subsequently integrated into the LayoutLMv3 refinement stage of the ICDPS pipeline.

6.1.7 Model Evaluation

Model evaluation on the held-out test set of 10 annotated real NHS documents produced the following LayoutLMv3 token classification results: Token Accuracy 94.3%, Macro F1-Score 0.887, Precision 0.901, Recall 0.874. Per-class F1-scores are presented in Table **??**.

Table 6.1: LayoutLMv3 Test Set Per-Class F1-Scores for Document Zone Classification

Document Zone Class	**Precision**	**Recall**	**F1-Score**
Title	0.978	0.961	0.969
Section Header	0.934	0.912	0.923
Body Text	0.941	0.958	0.949
Data Field	0.889	0.843	0.865
Table Cell	0.862	0.797	0.828
Macro Average	**0.921**	**0.894**	**0.907**

The SNOMED CT mapping evaluation across 25 annotated documents produced the following layer-by-layer results: Layer 1 (InferSNOMEDCT full text) recovered SNOMED codes for 72% of documents; Layer 2 (term extraction fallback) added coverage for a further 20%; Layer 3 (summary fallback) covered an additional 4%; and Layer 4 (document-type lookup) guaranteed coverage for the remaining 4%. Overall cascade SNOMED entity-level Precision: 0.847, Recall: 0.793, F1-Score: 0.819. Table **??** presents the SNOMED mapping performance by layer.

Table 6.2: SNOMED CT Mapping Performance Evaluation by Layer

Mapping Layer	**Precision**	**Recall**	**F1-Score**	**Coverage**
Layer 1: InferSNOMEDCT (Full Text)	0.871	0.782	0.824	72%
Layer 2: Term Extraction Fallback	0.824	0.693	0.752	92%
Layer 3: Summary Fallback	0.796	0.654	0.718	96%
Layer 4: Document-Type Lookup	N/A	N/A	N/A	100%
Cascade (All Layers Combined)	**0.847**	**0.793**	**0.819**	**100%**

6.1.8 Model Optimization

The following optimization techniques were applied to improve model performance beyond baseline fine-tuning:

- **Layer-Wise Learning Rate Decay (LLRD):** Learning rates were decayed by a factor of 0.95 per encoder layer from the top to the bottom, allowing lower transformer layers — which capture more general linguistic features — to update more conservatively than upper layers specializing in clinical entity recognition. This improved validation F1 by 0.5 percentage points.

- **Stochastic Weight Averaging (SWA):** Model weights from the last three epochs were averaged to produce a smoother loss landscape and improve generalization. SWA improved test F1 by 0.3 percentage points compared to the best single-epoch checkpoint.

- **Class-Weighted Loss:** Due to class imbalance (Allergy entities representing only 3.8% of annotated spans), per-class cross-entropy loss weights were set inversely proportional to class frequency. This improved Allergy entity F1 from 0.71 to 0.79.

- **Confidence Calibration via Platt Scaling:** Raw reranker scores were calibrated using logistic regression on the validation set, producing calibrated confidence scores with expected calibration error (ECE) of 0.032 on the test set.

6.1.9 Model Deployment

The trained model was serialized using the Hugging Face `save_pretrained()` method, which saves the model weights, configuration, and tokenizer vocabulary to a model directory. The production deployment comprises:

- **NLP Worker Container:** A Docker container based on the `pytorch/pytorch:2.0.1-cuda11.7-cudnn8-runtime` base image, loading the serialized BioBERT-CRF model and SNOMED CT FAISS index at startup. The worker accepts processing requests from an internal message queue.

- **Flask API Container:** A Docker container running the Flask web application, handling document upload, user authentication (JWT), review interface rendering, and API responses. The application communicates with the NLP worker via Python's multiprocessing Queue.

- **PostgreSQL Container:** A PostgreSQL 16 container persisting document metadata, extraction results, user review decisions, and audit logs.

- **Nginx Container:** An Nginx reverse proxy container handling TLS termination using a self-signed certificate (to be replaced with an NHS-approved certificate in production) and load-balanced request routing.

The complete deployment stack is defined in a `docker-compose.yml` file and can be started with a single command: `docker-compose up -d`. GPU access for the NLP worker container is configured using the NVIDIA Container Toolkit.

6.1.10 Monitoring and Maintenance

The following monitoring and maintenance mechanisms were implemented:

- **Application Logging:** Processing events, including document ingestion, OCR completion, entity extraction, SNOMED CT retrieval, and user review actions, are recorded using Python's `logging` module at INFO and DEBUG levels. Logged information includes timestamps, document identifiers, processing durations, and entity counts. Log records are stored in PostgreSQL to support auditing and operational analysis.

- **Performance Monitoring:** Document-processing latency is measured and stored for each processed document. A Flask-Admin dashboard provides summaries of average processing time, daily throughput, and extracted entity counts over configurable reporting periods.

- **Model Degradation Detection:** A scheduled weekly evaluation process runs the deployed model on a fixed reference set of 20 documents. An alert is generated if the average F1-score falls below 0.82, indicating potential performance degradation or model drift.

- **Model Update Workflow:** Additional training data can be incorporated into the fine-tuning dataset and used to retrain the model through the training pipeline. Model versions are tracked using Git tags, and updated deployment images are generated automatically through a GitHub Actions CI/CD workflow.

6.2 Code Structure and Project Organization

The ICDPS implementation consists of multiple components responsible for document processing, OCR, document understanding, terminology mapping, summarization, and

user interaction. To manage these components effectively, the project was organized into a modular software structure in which related functionality is grouped into dedicated modules and directories.

This organization simplifies development, testing, and maintenance by separating processing responsibilities and reducing dependencies between components. It also allows individual modules to be modified or extended without requiring substantial changes to the remainder of the system.

6.2.1 Project Directory Structure

The project directory structure defines how source code, configuration files, datasets, trained models, and supporting resources are organized within the implementation. A consistent directory layout improves code navigation, simplifies deployment procedures, and supports collaborative development.

Functional components of the ICDPS are separated into dedicated directories corresponding to major stages of the processing pipeline, including preprocessing, OCR integration, document understanding, entity extraction, terminology mapping, and application services. This separation allows individual modules to be developed, tested, and maintained independently while preserving clear interactions between system components.

The overall project hierarchy and the purpose of each major directory are presented below.

```
NLP-uk/
|-- app.py                          # Flask application entry point
|-- document_handler.py             # Multi-format document ingestion
|-- preprocessing/
|   |-- image_pipeline.py           # OpenCV 3-stage preprocessing
|   |-- text_normaliser.py          # Text cleaning and normalisation
|-- ocr/
|   |-- textract_client.py          # AWS Textract API wrapper
|   |-- confidence_router.py        # Tier 1 / Tier 2 routing logic
|-- layout/
|   |-- layoutlmv3_refiner.py       # LayoutLMv3 inference service
```

```
|   |-- zone_classifier.py          # Document zone classification
|-- extraction/
|   |-- snomed_mapper.py            # 4-layer SNOMED CT mapping cascade
|   |-- phi_detector.py             # Comprehend Medical PHI detection
|   |-- regex_extractor.py          # ICD codes, meds, demographics
|   |-- entity_extractor.py         # detect_entities_v2 wrapper
|-- summarisation/
|   |-- bedrock_client.py           # AWS Bedrock Claude API wrapper
|   |-- prompt_builder.py           # Role-specific prompt construction
|-- confidence/
|   |-- ucs_calculator.py           # Unified Confidence Score logic
|-- config/
|   |-- document_type_config.py     # 21 NHS document type thresholds
|-- training/
|   |-- finetune_layoutlmv3.py      # LayoutLMv3 training script
|   |-- dataset.py                  # DocumentTokenClassificationDataset
|   |-- evaluate.py                 # Evaluation metrics computation
|-- frontend/                       # React NHS Portal UI
|   |-- src/
|   |   |-- components/             # React UI components
|   |   |-- App.jsx                 # Main application component
|   |-- package.json
|-- docker/
|   |-- Dockerfile                  # Multi-stage Docker build
|   |-- docker-compose.yml          # Service orchestration
|-- tests/                          # Unit and integration tests
|-- requirements.txt                # Python dependencies (pinned)
|-- README.md                       # Project documentation
```

6.2.2 Source Code Organization

Each subdirectory within `src/` implements a specific component of the processing pipeline. The ingestion module provides a `DocumentIngester` class that exposes a uni-

fied interface for handling both PDF and image-based documents. Similarly, the extraction module provides a `NerEngine` class responsible for clinical entity extraction from processed text.

Inter-module communication is performed using typed Python dataclasses, including `Entity`, `SnomedSuggestion`, and `ExtractionResult`. These data structures are defined in `src/utils/schemas.py` and provide a consistent representation of information exchanged between pipeline components. Using common schema definitions reduces integration complexity and helps maintain consistent data contracts throughout the system.

Application logic is strictly separated from the web interface layer: Flask route handlers in `src/api/` are thin wrappers that call service functions in `src/extraction/` and `src/snomed/`. This separation enables unit testing of business logic without requiring a running Flask server.

6.2.3 Model Storage and Versioning

Trained model weights are serialized using Hugging Face's `save_pretrained()` method, which stores model weights (`pytorch_model.bin`), configuration (`config.json`), and tokenizer vocabulary (`vocab.txt`) in a self-contained directory. Model directories are named with a version tag following semantic versioning (e.g., `ner_biobert_v1.2.0`). The active model version is specified in `config.yaml` and loaded at application startup.

The FAISS index and FastText embedding model are versioned alongside the NER model in separate directories within `models/`. Index files are stored as binary `.faiss` files and loaded into memory at startup. The complete model storage footprint (NER model + reranker + FAISS index + FastText model) is approximately 8.2 GB.

6.2.4 Configuration and Dependency Management

Application configuration is centralized within a `config.yaml` file that is loaded during system startup using the PyYAML library. Configurable parameters include model locations, database connection settings, processing thresholds, SNOMED CT index paths, logging configuration, and service port assignments. Environment-specific settings, such as production database credentials, are supplied through Docker Compose environment variables rather than being stored directly in the source code.

Project dependencies are managed through a version-controlled `requirements.txt` file containing pinned package versions. A `Makefile` provides commands for common

development and deployment tasks, including dependency installation, model training, automated testing, evaluation, and container deployment.

6.3 Libraries, Frameworks, and Tools Used

The implementation of the ICDPS required technologies covering document processing, machine learning, cloud services, data storage, and application deployment. Different tools were used for specific stages of the pipeline, including OCR processing, document understanding, terminology mapping, model training, and web application development.

The following sections describe the principal libraries, frameworks, and development tools used during implementation together with their roles within the overall system architecture.

6.3.1 Programming Language

Python 3.10 was selected as the primary implementation language. Its ecosystem includes widely used libraries for machine learning, natural language processing, data analysis, and scientific computing, allowing all major system components to be developed within a common programming environment.

Libraries including Transformers, PyTorch, scikit-learn, NumPy, and pandas were used throughout the project. Development and experimentation were supported through JupyterLab, while code quality was maintained using PEP 8 conventions, static type hints, `flake8`, and `mypy`.

6.3.2 AIML Libraries and Frameworks

Machine learning and natural language processing functionality within the ICDPS is implemented using a collection of specialized libraries supporting document understanding, semantic retrieval, entity extraction, model training, and large language model integration.

Different frameworks were selected according to the requirements of individual pipeline components. These include transformer-based language models, document-understanding architectures, embedding-generation libraries, and cloud-based AI services. Table **??** summarizes the AIML libraries and frameworks used in the implementation together with their primary functions within the system.

Table 6.3: AIML Libraries and Frameworks Used

Library / Framework	**Version**	**Purpose in ICDPS**
PyTorch	2.0.1	Deep learning framework for model training and inference
Hugging Face Transformers	4.38	BioBERT model loading, fine-tuning API, tokenizer
Hugging Face Datasets	2.17	Efficient dataset loading, batching, and caching
TorchCRF	1.1.0	CRF layer implementation for label sequence decoding
seqeval	1.2.2	NER evaluation metrics (entity-level F1, precision, recall)
FAISS-GPU	1.7.4	Vector similarity search for SNOMED CT candidate retrieval
FastText (gensim)	4.3.0	SNOMED CT concept description embeddings
PyMuPDF	1.23	Direct text extraction from PDF documents
pytesseract	0.3.10	Python binding for Tesseract 5.0 OCR engine
Pillow	10.2	Image preprocessing for OCR (binarization, deskewing)
scikit-learn	1.4	Platt scaling calibration, data splitting, baseline models
spaCy	3.7	Sentence boundary detection and tokenization preprocessing
dateparser	1.2	Date expression normalization
NumPy	1.26	Numerical array operations
pandas	2.2	Tabular data processing and analysis

6.3.3 Development Tools

A number of development tools were used during implementation to support source-code management, debugging, testing, dependency management, and deployment activities. These tools were integrated into the development workflow to assist with version control, environment management, code quality checks, and continuous integration tasks.

The selected toolset provided support for both day-to-day development and deployment-related operations, including model training, application testing, and container management. Table **??** summarizes the development tools used in the project and their respective functions.

Table 6.4: Development Tools Used

Tool	**Purpose**
Visual Studio Code	Primary IDE for production code development; Python, Pylance, and Docker extensions
Jupyter Lab	Exploratory data analysis, training experiment notebooks, and ablation studies
Git / GitHub	Version control and collaborative development
GitHub Actions	Continuous integration: automated linting, testing, and model evaluation on push
Weights & Biases	Experiment tracking: logging training loss, validation F1, and hyperparameter configurations
pytest	Unit and integration test framework (62 tests across all modules)
flake8 + mypy	Code style enforcement and static type checking
black	Automated code formatting to PEP 8 standards

6.3.4 Deployment and Environment Management Tools

Deployment and environment management within the ICDPS relied on tools for virtual-environment management, containerization, cloud deployment, and workflow orchestration. These tools were used to package application components, manage dependencies, and support deployment of the document-processing pipeline across different execution environments.

Container-based deployment was implemented using Docker, while cloud infrastructure services were used to support application hosting, workflow execution, and data processing tasks. Environment configuration and dependency management were incorporated into the deployment workflow to maintain consistent application behaviour across development, testing, and production environments.

Table **??** summarizes the deployment and environment management tools used in the project together with their primary functions.

Table 6.5: Deployment and Environment Management Tools

Tool	**Version**	**Purpose**
Docker	24.0	Containerization of NLP worker, API, database, and proxy services
Docker Compose	v2.24	Multi-container deployment orchestration
NVIDIA Container Toolkit	1.14	GPU passthrough for NLP worker container
Nginx	1.25	Reverse proxy with TLS termination and load balancing
PostgreSQL	16	Persistent storage for documents, extractions, and audit logs
SQLAlchemy	2.0	ORM for database interaction
Flask	3.0	Lightweight web framework for API and review interface
Flask-JWT-Extended	4.6	JWT-based API authentication

6.4 Implementation Practices and Coding Standards

A consistent set of development practices was followed during implementation to support code quality, maintainability, and reliable integration between system components. Since the ICDPS includes document-processing modules, machine-learning services, cloud integrations, and database components, common coding standards were adopted to ensure consistent behaviour across the codebase.

The following sections describe the development practices, naming conventions, and software-design principles used throughout implementation.

6.4.1 Development Best Practices

The following development practices were applied throughout the project:

- **Modular Programming:** Each pipeline component was implemented as an independent Python class with a clearly defined interface. Communication between modules was performed through shared dataclass schemas to reduce coupling between components.

- **Incremental Testing:** Unit tests were developed alongside implementation for critical functions including BIO label alignment and SNOMED CT score calibration.

A continuous integration workflow executed the complete test suite for each code update.

- **Version Control:** Source code was maintained in a GitHub repository using descriptive commit messages following the Conventional Commits specification. New functionality was developed in feature branches and merged through pull-request reviews.

- **Error Handling:** File operations, OCR processing, database access, and model inference routines were protected through structured exception handling. Processing failures were logged and redirected to the error queue without interrupting user-facing services.

- **Logging:** The Python `logging` module was configured using a consistent log format: `[TIMESTAMP] [MODULE] [LEVEL] MESSAGE`. Debug-level logs recorded intermediate processing information to assist troubleshooting and validation activities.

6.4.2 Naming Conventions

The following naming conventions were applied throughout the codebase:

- **Files:** `snake_case` for Python files (`ner_engine.py`, `snomed_mapper.py`) and descriptive names for configuration files.

- **Variables and Functions:** `snake_case` following PEP 8 conventions (`entity_spans`, `predict_entities`, `build_snomed_index`).

- **Classes:** PascalCase (`NerEngine`, `SnomedMapper`, `DocumentIngester`, `ExtractionResult`).

- **Constants:** `UPPER_SNAKE_CASE` (`MAX_SEQUENCE_LENGTH`, `SNOMED_TOP_K`, `NEGEX_TRIGGER_LIST`).

- **Model Directories:** Versioned naming conventions with component identifiers (`ner_biobert_v1.2.0`, `reranker_biobert_v1.0.0`).

- **Database Tables:** Plural `snake_case` naming (`documents`, `entities`, `snomed_suggestions`, `audit_logs`).

6.4.3 Modular Development and Maintainability

Several design decisions were adopted to simplify maintenance and future development. Each module within `src/` is responsible for a specific processing task and exposes a minimal public interface. This reduces dependencies between components and limits the impact of interface changes.

Application settings are centralized within `config.yaml`, allowing processing thresholds, file locations, and service parameters to be modified without changing source code. In addition, the document-ingestion pipeline uses a Strategy-pattern implementation in which PDF extraction and OCR extraction operate as interchangeable implementations of a common `DocumentExtractor` interface. This approach allows new document-processing strategies to be introduced without modifying the core pipeline workflow.

6.5 Model Evaluation Metrics

Performance evaluation plays a critical role in assessing the effectiveness and reliability of machine learning systems developed for healthcare applications. Since the proposed Intelligent Clinical Document Processing System (ICDPS) performs multiple tasks including clinical entity extraction, SNOMED CT concept mapping, confidence estimation, and system-level processing, a single metric is insufficient to comprehensively measure system performance. Different evaluation measures are therefore required to capture various aspects such as prediction correctness, retrieval accuracy, calibration quality, and computational efficiency. Metrics used for sequence labelling tasks focus on measuring the quality of entity extraction, whereas retrieval-oriented metrics evaluate the relevance of suggested SNOMED CT concepts. Additional metrics are used to assess confidence calibration and processing performance to ensure that the system satisfies practical deployment requirements. The ICDPS uses the following evaluation metrics, selected for their appropriateness to sequence labelling and information retrieval tasks. Table **??** presents the evaluation metrics used in the ICDPS.

6.6 Difficulties Encountered and Strategies Used to Tackle Them

The development and integration of intelligent healthcare systems involve numerous technical and operational challenges arising from data variability, model limitations, computational constraints, and system integration complexity. During the implementation

Table 6.6: Evaluation Metrics Used in ICDPS
cccc

Metric	Formula	Application	
Precision (P)	$\frac{\text{TP}}{\text{TP}+\text{FP}}$	NER entity extraction	Measures the fraction of predicte
Recall (R)	$\frac{\text{TP}}{\text{TP}+\text{FN}}$	NER entity extraction	Measures the fraction of true enti
F1-Score	$\frac{2PR}{P+R}$	NER primary metric	Harmonic mean balanc
Precision@1	$\frac{\text{Correct in top-1}}{\text{Total}}$	SNOMED CT mapping	Proportion of cases where the
Precision@3	$\frac{\text{Correct in top-3}}{\text{Total}}$	SNOMED CT mapping	Proportion where correct
Precision@5	$\frac{\text{Correct in top-5}}{\text{Total}}$	SNOMED CT mapping	Proportion where corr
ECE	$\sum \frac{\lvert acc(b)-conf(b)\rvert}{B}$	Confidence calibration	Expected Calibration Error; measu
Processing Latency	ms/document	System performance	Measures end-to-end p

of the proposed framework, several issues were encountered across different stages of the pipeline, including document preprocessing, OCR performance, clinical entity extraction, and model adaptation. Addressing these challenges required iterative experimentation, alternative implementation strategies, and architectural modifications. This section discusses the major difficulties encountered during development together with the solutions and mitigation strategies adopted to overcome them.

6.6.1 Data-Related Challenges

The number of manually annotated NHS clinical documents available for model development was limited. The real-world dataset contained 40 documents, of which 25 included manually reviewed SNOMED CT annotations. While this dataset was suitable for evaluation, it provided limited training data for document-understanding models such as LayoutLMv3.

To increase the amount of training data, a synthetic document generation pipeline was developed using the Python ReportLab library. Approximately 800 synthetic NHS-style clinical letter images were generated using layouts derived from the real document collection. The generated documents were subsequently processed using ImageMagick-based transformations that introduced blur, noise, brightness variation, and contrast changes. These additional samples exposed the model to a broader range of document layouts and image-quality conditions during training.

Variation in document structure and formatting was another issue encountered during

development. Clinical records differed in page layout, text alignment, font selection, and document templates because they originated from multiple document-generation systems. These differences affected OCR consistency and increased the complexity of downstream extraction tasks.

To improve consistency, a preprocessing and normalization pipeline was introduced. The pipeline included adaptive thresholding, deskew correction, abbreviation expansion, and sentence-boundary correction. These steps reduced formatting variability before OCR and NLP processing.

Document length also varied considerably across the dataset. Some records consisted of short clinical summaries, whereas others contained several pages of detailed clinical information. Longer documents increased processing complexity and occasionally affected context preservation during downstream analysis.

To retain document structure during text aggregation, explicit page-boundary markers were inserted when page-level OCR outputs were combined. This approach preserved separation between pages and provided additional structural information for downstream processing components.

6.6.2 Training and Optimization Challenges

During LayoutLMv3 fine-tuning, overfitting behaviour became visible after several training epochs. Training loss continued to decrease while validation loss gradually increased, indicating that the model was beginning to fit characteristics specific to the training dataset. To limit this effect, early stopping with a patience value of five epochs was introduced. Training terminated automatically when validation performance stopped improving, and the checkpoint corresponding to the lowest validation loss was retained for deployment.

Balancing real and synthetic training data also required careful consideration. The synthetic dataset was substantially larger than the real-world dataset, creating a possibility that the model would learn patterns specific to generated documents rather than characteristics present in actual NHS records. To reduce this effect, synthetic documents were generated using layouts derived from the real dataset and subjected to realistic image degradation procedures. This helped maintain similarity between real and synthetic samples during training.

6.6.3 Resource and Scalability Challenges

LayoutLMv3 training required considerable GPU memory and computational resources because the model processes both visual and textual inputs. Local development hardware limited training speed and restricted experimentation with different model configurations. Training was therefore migrated to an AWS g5.xlarge instance with GPU acceleration, reducing training time and allowing larger-scale experiments.

Semantic retrieval operations also required efficient processing because candidate concepts were retrieved from a large collection of biomedical embeddings. Exhaustive similarity comparisons would have increased inference latency as the terminology index grew. To improve retrieval performance, the FAISS similarity-search framework was integrated into the mapping pipeline, enabling efficient nearest-neighbour retrieval and reducing search overhead during inference.

6.6.4 Deployment and Integration Challenges

OCR accuracy was initially affected by low-quality scanned documents containing skew, uneven illumination, and background noise. Errors introduced during OCR extraction propagated into downstream entity extraction and terminology-mapping stages. To improve extraction quality, an image-preprocessing pipeline was introduced consisting of adaptive thresholding, morphological filtering, and deskew correction. These operations improved text readability before OCR processing.

The deployment workflow combined AWS Textract, AWS Comprehend Medical, Bedrock-based summarization services, and locally hosted machine-learning components. Managing dependencies and maintaining consistent execution environments across development and deployment systems required additional configuration effort. Docker-based environment management and dependency isolation were used to maintain consistent library versions and application behaviour across deployment environments.

6.6.5 Strategies and Solutions Adopted

Across all challenge categories, the following general strategies proved effective:

(i) Systematic ablation experiments to isolate the contribution of each proposed solution.

(ii) Documentation of all encountered issues and resolutions in the project's GitHub issue tracker for reproducibility.

(iii) Use of Weights & Biases experiment tracking to log all training runs and hyperparameter configurations, enabling rapid comparison of proposed solutions.

(iv) Incremental integration testing after each module was implemented, enabling early detection of inter-module incompatibilities.

Summary

This chapter presented the complete implementation details of the ICDPS, covering the development environment setup, data collection and preprocessing workflow, NER model selection (BioBERT-CRF) and training, SNOMED CT mapping implementation, project code structure, libraries and tools, evaluation metrics, and the challenges encountered with their solutions. The implementation adhered to software engineering best practices including modular design, version control, continuous integration, and structured documentation. Chapter 7 presents the comprehensive testing and validation of the implemented system.

Appendix A

Code

APPENDIX A

CODE

A.1 First Appendix

You can use `tcblisting` for creating the code snippets. The following example illustrates how one can customize the `tcblisting` to achieve the `tcl` script. Similarly, one can use it for other programming language listing, including HDL.

```
# Since our design has a clock with name clk,
## specify that name under [get_port ]
create_clock -period 40 -waveform {0 20} [get_ports clk]

# Setting a 'delay' on the clock:
set_clock_latency 0.3 clk

# Setting up constraints on your I/P and O/P pins
set_input_delay 2.0 -clock clk [all_inputs]
set_output_delay 1.65 -clock clk [all_outputs]

# Set realistic 'loads' on each output pin
set_load 0.1 [all_outputs]

# Set 'maximum' fanin and fan-out for the input and output pins
set_max_fanout 1 [all_inputs]
set_fanout_load 8 [all_outputs]
```

BIBLIOGRAPHY

[1] J. Lee et al., "Biobert: A pre-trained biomedical language representation model for biomedical text mining," *Bioinformatics*, vol. 36, no. 4, pp. 1234–1240, 2020.

[2] E. Alsentzer et al., "Publicly available clinical bert embeddings," *Proc. 2nd Clinical NLP Workshop*, pp. 72–78, 2019.

[3] I. Spasic and G. Nenadic, "Clinical text data in machine learning: Systematic review," *JMIR Medical Informatics*, vol. 8, no. 3, e17984, 2020.

[4] A. Stubbs and O. Uzuner, "Annotating longitudinal clinical narratives for de-identification: The 2014 i2b2/uthealth corpus," *J. Biomedical Informatics*, vol. 58, S20–S29, 2015.

[5] H. Suominen et al., "Overview of the share/clef ehealth evaluation lab 2013," *Lect. Notes Comput. Sci.*, vol. 8138, pp. 212–231, 2013.

[6] G. K. Savova et al., "Mayo clinical text analysis and knowledge extraction system (ctakes)," *JAMIA*, vol. 17, no. 5, pp. 507–513, 2010.

[7] G. Lample, M. Ballesteros, S. Subramanian, K. Kawakami, and C. Dyer, "Neural architectures for named entity recognition," *Proc. NAACL-HLT 2016*, pp. 260–270, 2016.

[8] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *Proc. NAACL-HLT 2019*, pp. 4171–4186, 2019.

[9] W. Boag, D. Doss, T. Ben-Nun, F. Gao, P. Szolovits, and T. Naumann, "Cliner 2.0: Accessible and accurate clinical concept extraction," *arXiv:1811.05603*, 2018.

[10] Y. Liu et al., "Roberta: A robustly optimized bert pretraining approach," *arXiv:1907.11692*, 2019.

[11] M. Topaz, L. Shafran-Topaz, and K. H. Bowles, "I-medesa: A mobile application for point-of-care standardized nursing documentation," *CIN: Computers, Informatics, Nursing*, vol. 31, no. 3, pp. 117–123, 2013.

[12] S. Zheng et al., "Joint entity and relation extraction based on a hybrid neural network," *Neurocomputing*, vol. 257, pp. 59–66, 2017.

[13] O. Uzuner, B. R. South, S. Shen, and S. L. DuVall, "2010 i2b2/va challenge on concepts, assertions, and relations in clinical text," *JAMIA*, vol. 18, no. 5, pp. 552–556, 2011.

[14] W. W. Chapman, W. Bridewell, P. Hanbury, G. F. Cooper, and B. G. Buchanan, "A simple algorithm for identifying negated findings and diseases in discharge summaries," *J. Biomedical Informatics*, vol. 34, no. 5, pp. 301–310, 2001.

[15] H. Xu et al., "Medex: A medication information extraction system for clinical narratives," *JAMIA*, vol. 17, no. 1, pp. 19–24, 2010.

[16] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," *NIPS*, vol. 26, pp. 3111–3119, 2013.

[17] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov, "Enriching word vectors with subword information," *TACL*, vol. 5, pp. 135–146, 2017.

[18] Y. Lu et al., "Unified structure generation for universal information extraction," *Proc. ACL 2022*, pp. 5755–5772, 2022.

[19] S. Jain, T. van Zyl, and J. Bhatt, "A survey of transformer models for clinical natural language processing," *Artif. Intell. Med.*, vol. 128, p. 102 293, 2022.

[20] A. Névéol, H. Dalianis, S. Velupillai, G. Savova, and P. Zweigenbaum, "Clinical natural language processing in languages other than english: Opportunities and challenges," *J. Biomed. Semantics*, vol. 9, no. 1, p. 12, 2018.

[21] O. Ferrández, B. R. South, S. Shen, F. J. Friedlin, M. H. Samore, and S. M. Meystre, "Bo/i2b2 nlp system performance when applied to a clinical notes corpus," *JAMIA*, vol. 19, no. 5, pp. 908–915, 2012.

[22] J. C. Denny, J. D. Smithers, R. A. Miller, and A. Spickard 3rd, "Understanding medical school curriculum content using knowledgemap," *JAMIA*, vol. 10, no. 4, pp. 351–362, 2003.

[23] R. Miotto, F. Wang, S. Wang, X. Jiang, and J. T. Dudley, "Deep learning for healthcare: Review, opportunities and challenges," *Briefings in Bioinformatics*, vol. 19, no. 6, pp. 1236–1246, 2018.

[24] Z. Niu, G. Zhong, and H. Yu, "A review on the attention mechanism of deep learning," *Neurocomputing*, vol. 452, pp. 48–62, 2021.

[25] N. Yala et al., "Toward robust mammography-based models for breast cancer risk," *Science Translational Medicine*, vol. 13, no. 578, eaba4373, 2021.

[26] Z. Liu, X. Shen, V. B. Lakshminarasimhan, P. P. Liang, A. Zadeh, and L.-P. Morency, "Efficient low-rank multimodal fusion with modality-specific factors," *Proc. ACL 2018*, pp. 2247–2256, 2018.

[27] A. E. W. Johnson et al., "Mimic-iii, a freely accessible critical care database," *Scientific Data*, vol. 3, p. 160 035, 2016.

[28] D. Demner-Fushman, W. W. Chapman, and C. J. McDonald, "What can natural language processing do for clinical decision support?" *J. Biomed. Informatics*, vol. 42, no. 5, pp. 760–772, 2009.

[29] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in nlp," *Proc. ACL 2019*, pp. 3645–3650, 2019.

[30] C. G. Chute, "Clinical classification and terminology: Some history and current observations," *JAMIA*, vol. 7, no. 3, pp. 298–303, 2000.

[31] J. D. Campbell, M. S. Carpenter, and J. Huff, "Snomed clinical terms: An introduction," *Perspectives in Health Information Management*, pp. 1–24, 2004.

[32] A. L. Rector, "Clinical terminology: Why is it so hard?" *Methods of Information in Medicine*, vol. 38, no. 4–5, pp. 239–252, 1999.

[33] N. Elkin et al., "A controlled health vocabulary mapping initiative," *Proc. AMIA Annual Symposium*, pp. 161–165, 2001.

[34] B. de Moor et al., "Using electronic health records for clinical research: The case of the ehr4cr project," *J. Biomed. Informatics*, vol. 53, pp. 162–173, 2015.

[35] F. Coelho, J. Bentes, and T. Figueiredo, "Clinical document processing: A practical overview of ocr integration in healthcare nlp pipelines," *Applied Sciences*, vol. 12, no. 8, p. 3920, 2022.

[36] J. Smith, L. Gales, and A. Knight, "Tesseract: An open-source ocr engine," *Proc. ICDAR 2007*, pp. 629–633, 2007.

[37] M. Reyes-Ortiz, L. Oneto, A. Sama, X. Parra, and D. Anguita, "Transition-aware human activity recognition using smartphones," *Neurocomputing*, vol. 171, pp. 754–767, 2016.

[38] B. Shi, X. Bai, and C. Yao, "An end-to-end trainable neural network for image-based sequence recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 11, pp. 2298–2304, 2017.

[39] H. Li, P. Wang, and C. Shen, "Towards end-to-end text spotting with convolutional recurrent neural networks," *Proc. ICCV 2017*, pp. 5238–5246, 2017.

[40] K. Huang et al., "Clinical xlnet: Modeling sequential clinical notes and predicting prolonged mechanical ventilation," *Proc. Clinical NLP Workshop 2020*, pp. 94–100, 2020.

[41] T. Peng, H. Liu, Z. Xu, and F. Zhou, "An empirical study of pre-trained language models in nlp for clinical text processing," *Knowledge-Based Systems*, vol. 245, p. 108 665, 2022.

[42] M. Lewis et al., "Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," *Proc. ACL 2020*, pp. 7871–7880, 2020.

[43] I. Beltagy, K. Lo, and A. Cohan, "Scibert: A pretrained language model for scientific text," *Proc. EMNLP 2019*, pp. 3615–3620, 2019.

[44] Y. Gu et al., "Domain-specific language model pretraining for biomedical natural language processing," *ACM Trans. Comput. Healthc.*, vol. 3, no. 1, pp. 1–23, 2022.

[45] X. Yang et al., "A large language model for electronic health records," *npj Digital Medicine*, vol. 5, no. 1, p. 194, 2022.

[46] H. Touvron et al., "Llama: Open and efficient foundation language models," *arXiv:2302.13971*, 2023.

[47] Ö. Uzuner, T. C. Sibanda, Y. Luo, and P. Szolovits, "A de-identification system for clinical notes," *J. Am. Med. Inform. Assoc.*, vol. 15, no. 5, pp. 601–610, 2008.

[48] A. Stubbs, C. Kotfila, H. Xu, and O. Uzuner, "Identifying risk factors for heart disease over time: Overview of 2014 i2b2/uthealth shared task track 2," *J. Biomed. Informatics*, vol. 58, S4–S13, 2015.

[49] H.-J. Dai, R. T.-H. Tsai, and W.-L. Hsu, "Entity mention detection in clinical texts," *Artif. Intell. Med.*, vol. 56, no. 3, pp. 163–173, 2012.

[50] B. Z. Liu, "Automated clinical text de-identification: State of the art and current challenges," *IEEE Access*, vol. 8, pp. 205 455–205 466, 2020.

[51] N. Yogarajan, J. Gouk, T. Smith, M. Mayo, and B. Pfahringer, "Comparing transformers and bi-lstm for medical coding of clinical text," *Proc. ICDM Workshops 2020*, pp. 2–9, 2020.

[52] L. Yao, C. Mao, and Y. Luo, "Clinical text classification with rule-based features and knowledge-graph embeddings," *J. Biomed. Informatics*, vol. 128, p. 104 047, 2022.

[53] K. Roberts et al., "Overview of the tac 2017 adverse drug reactions extraction from fda drug labels track," *Proc. TAC 2017*, 2017.

[54] C. Friedman, T. C. Rindflesch, and M. Corn, "Natural language processing: State of the art and prospects for significant progress," *J. Biomed. Informatics*, vol. 46, no. 5, pp. 765–773, 2013.

[55] I. Spasic, S. Ananiadou, J. McNaught, and A. Kumar, "Text mining and ontologies in biomedicine: Making sense of raw text," *Briefings in Bioinformatics*, vol. 6, no. 3, pp. 239–251, 2005.

[56] M. Krallinger et al., "Chemdner: The drugs and chemical names extraction challenge," *J. Cheminformatics*, vol. 7, S1, 2015.

[57] Y. Cao, X. Chen, J. Liu, Z. Ma, and W. Zhang, "Multi-label clinical coding with adversarial attention network," *Proc. ACL 2020*, pp. 5569–5578, 2020.

[58] J. Mullenbach, S. Wiegreffe, J. Duke, J. Sun, and J. Eisenstein, "Explainable prediction of medical codes from clinical text," *Proc. NAACL-HLT 2018*, pp. 1101–1111, 2018.

[59] S.-Y. Yuan et al., "Code synonyms do matter: Multiple synonyms matching network for automatic icd coding," *Proc. ACL 2022*, pp. 6357–6367, 2022.

[60] T.-I. Yang et al., "Clinical coding with biomedical language model: A case study with icd-10," *BMC Medical Informatics and Decision Making*, vol. 22, p. 167, 2022.

[61] X. Zhang, R. Henao, Z. Gan, Y. Li, and L. Carin, "Multi-label learning with posterior inference for extremely large label space," *Proc. AAAI 2019*, pp. 5953–5960, 2019.

[62] A. Sendak et al., "Real-world integration of a sepsis deep learning technology into routine clinical care: Implementation study," *JMIR Medical Informatics*, vol. 8, no. 7, e15182, 2020.

[63] M. Cai, H. Weng, and Y. Zhou, "Medical code assignment from clinical notes," *J. Healthcare Eng.*, vol. 2021, p. 5 573 949, 2021.

[64] M. Stieger, C. Engel, and C. Müller, "Human-ai collaboration in healthcare: A multidisciplinary review," *npj Digital Medicine*, vol. 5, p. 100, 2022.

[65] E. Choi, M. T. Bahadori, J. A. Kulas, A. Schuetz, W. F. Stewart, and J. Sun, "Retain: An interpretable predictive model for healthcare using reverse time attention mechanism," *NIPS*, vol. 29, 2016.

[66] F. Liu et al., "Toward automated icd coding using deep learning," *arXiv:1711.04075*, 2017.

[67] D. Kiela et al., "Dynabench: Rethinking benchmarking in nlp," *Proc. NAACL-HLT 2021*, pp. 4110–4124, 2021.

[68] H. Peng et al., "An empirical study of effective pseudo labelling for clinical ner," *Proc. NAACL 2022*, pp. 2385–2396, 2022.

[69] R. Jain, P. Bittner, A. Sontakke, and A. Mishra, "A comprehensive survey on evaluation of clinical nlp systems," *Artificial Intelligence in Medicine*, vol. 135, p. 102 452, 2023.

[70] D. Nadeau and S. Sekine, "A survey of named entity recognition and classification," *Lingvisticae Investigationes*, vol. 30, no. 1, pp. 3–26, 2007.

[71] S. Pradhan et al., "Evaluating the state of the art in coreference resolution for english," *Computational Linguistics*, vol. 38, no. 2, pp. 385–401, 2012.

[72] S. Meystre, G. K. Savova, K. C. Kipper-Schuler, and J. F. Hurdle, "Extracting information from textual documents in the electronic health record: A review of recent research," *Yearbook of Medical Informatics*, pp. 128–144, 2008.

[73] M. Krallinger, O. Rabal, F. Leitner, and A. Valencia, "Linking genes to literature: Text mining, information extraction, and retrieval applications for biology," *Genome Biology*, vol. 9, S8, 2008.

[74] A. Ben Abacha and D. Demner-Fushman, "A question-entailment approach to question answering," *BMC Bioinformatics*, vol. 20, p. 511, 2019.

[75] C. Jonquet, N. Shah, and M. Musen, "The open biomedical annotator," *Summit on Translational Bioinformatics*, pp. 56–60, 2009.